

Two Portable Components for Meeting Hard Prediction Quotas in Deep Classifiers

Anonymous authors

Paper under double-blind review

Abstract

Deployed classifiers often face a hard cap on how many items of a known evaluation batch may be assigned to a constrained class. Examples include screening pools that can escalate only so many cases, sometimes with per-group limits. We call this a transductive count constraint. We study a training-time approach that satisfies this cap without editing predictions after training, and we identify which of its components drive the effect. The approach, which we call TraLO, combines a saturating penalty, a ratcheted-and-frozen multiplier, an Adam-state reset at first feasibility, and an undershoot hinge. Across a 1944-run grid on MedMNIST classification tasks (28×28 images upsampled to 224×224), constraint-time training reaches the cap natively in nearly all runs, improving overall classification quality by 1.6 to 5.3 percentage points over satisfying the cap after training. On quality of the constrained class itself, one regime shows a consistent advantage across all backbones tested: OctMNIST at tight caps, with the largest gains on a ViT (not itself a rare-class regime). The rest of the grid is a statistical tie, reported as such. Our central mechanistic finding is that this constrained-class advantage is carried by two portable components, the optimizer reset and the undershoot hinge. Both transfer as additions to alternative constraint-trained baselines and close most of the tight-cap gap. In contrast, the saturating penalty shape after which the package is named proves neutral by both ablation and graft. Native-resolution generalization is left to future work. We release the full regime map, including the tie region and the negative penalty-shape result, along with the per-run corpus.

1 Introduction

A surprising number of classification systems are deployed under a constraint not on which items get a label but on *how many* do. A screening program reads a fixed pool of scans and may escalate only so many of them to specialist review; an audit team can act on only so many flagged filings in a quarter. In each case the operational requirement is a *hard upper bound on the count* of predictions in a constrained class over the evaluation set. The items to be labeled are known in advance (a fixed batch to be scored, not a stream), so the constraint is naturally *transductive*: it is a property of the whole test-set prediction vector, not of any single example.

The standard way to honor such a cap is *post-hoc clipping*: train a normal model, rank the test items by their score for the constrained class, keep the top- K , and reassign the rest. This is trivial and always meets the count, so it is the realistic deployment baseline. But it meets the count by editing predictions, often dozens of them, each a forced error on an item the model was confident about, so accuracy degrades. The other end of the spectrum is *constrained training*: encode the cap as a constraint and solve the Lagrangian with dual ascent (Fioretto et al., 2020; Hounie et al., 2023; Chamon & Ribeiro, 2020). In principle this is exactly the right formulation, but for a hard count the dual multiplier of a linear penalty climbs without bound: on a one-epoch-warmup probe (App. A) it climbs steeply while a bounded multiplier barely moves, for the same final count. The implementations remain competitive and we do not over-read the escalation; it is a symptom, not the damage. What it marks is a formulation that on some cells never reaches feasibility at all (Sec. 5.4).

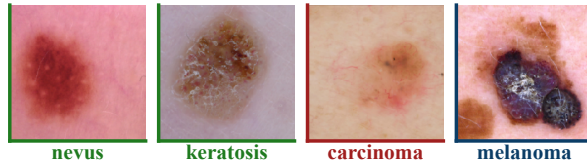


Figure 1: **DermMNIST lesions, benign to malignant** (green frame benign, red malignant, blue the *constrained* class). The task caps how many test images may be labeled melanoma, the minority high-stakes class (11.1% of the test set); the constraint-trained methods studied here meet that cap *during training* rather than clipping afterward.

This paper. We characterize a third answer that lands on the count by itself, without prediction surgery, and *we identify which of its ingredients actually do the work*. The package we study — for exposition we refer to it as TraLO (*Transductive Loss Optimization*) — combines a saturating transductive penalty (a rational-saturation term plus a bounded quadratic in the soft excess over the cap) with a multiplier that only moves upward while the count is violated and freezes the first time it is met, an Adam-state reset at first satisfaction, and an undershoot hinge that restores constrained-class recall once the cap is met. The package as a whole meets the count natively on essentially every run and beats post-hoc clipping on macro-F1 in nearly every regime. But the internal decomposition matters more than the brand: our ablation and graft show that the constrained-class win rests on *two* of these ingredients — the optimizer reset and the undershoot hinge — and that they transfer to the dual baselines, while the saturating penalty shape after which the package is named proves neutral.

Who should care. *Practitioners* building screening, audit, or triage pipelines under hard batch caps on MedMNIST-scale tasks: this paper reports when constraint-time training gains 1.6–5.3 percentage points of overall quality over satisfying the cap after training, at a compute cost of a few extra minutes per run. Whether the same margin holds at native imaging resolution is not addressed here. *Researchers on constrained deep learning:* this paper isolates two portable ingredients (an Adam-state reset at first feasibility and an undershoot hinge) that transfer across constraint-trained methods and, under matched training budgets, close most of the tight-cap gap between them; the shape of the enclosing penalty proves neutral by both ablation and graft.

Contributions.

- **An empirical characterization of when constrained training wins.** We formalize the operational “ $\leq K$ predictions of class c ” requirement as a transductive count constraint (Sec. 3) and map, over a 1944-run grid, the regimes where constraint-trained methods beat clipping on macro-F1 (nearly all of the grid; Sec. 5.3) and where they beat one another on constrained-class F1 (essentially: OctMNIST tight caps; Secs. 5.1–5.2).
- **Two portable components that carry the constrained-class advantage.** An ablation isolates an Adam-state reset at first feasibility and an undershoot hinge (App. B) as load-bearing; a cross-method graft (Sec. 6) shows they transfer to Fioretto-LDF (Fioretto et al., 2020) and Hounie-RCL (Hounie et al., 2023) and close most of the tight-cap gap. Neither component is derived; both are motivated empirically and identified by ablation.
- **The saturating penalty shape is quality-neutral.** The saturating penalty after which the package we study is named is neutral to constrained-class quality: swapping it for a linear penalty at the same ratcheted multiplier moves cc-F1 by -0.001 across 24 runs, with 6 runs landing on identical predictions (Sec. 6; App. B.2). Eq. (1) earns its place by keeping the frozen multiplier small and readable, not by earning the quality gain.

- **A public regime map and per-run corpus.** We release the full 1944-run grid (per-cell, per-seed) and the code, so any of the headline claims — including the negative and tie regions — is independently verifiable at the granularity we report them.

2 Related Work

Lagrangian and dual constrained learning. Encoding requirements as constraints solved with primal-dual methods is the classical recipe (Bertsekas, 1982). The idea was introduced to neural networks by Platt & Barr (1988) and brought to modern deep networks by Lagrangian-duality training (Fioretto et al., 2020), with PAC statistical footing (Chamon & Ribeiro, 2020; Chamon et al., 2023) and an end-to-end survey in Kotary et al. (2021). Resilient Constrained Learning (Hounie et al., 2023) adapts the constraint level to make hard requirements attainable, and constraints can beat fixed penalties on sparsity targets (Gallego-Posada et al., 2022). On the multiplier update, PID-style rules stabilize the multiplier in safe RL (Stooke et al., 2020) and, more recently, in supervised deep learning as PI controllers on the Lagrange multiplier (Sohrabi et al., 2024). Near-optimal solutions of constrained learning problems have been characterized for non-convex losses (Elenter et al., 2024), and open-source infrastructure for constrained deep learning is now available (Gallego-Posada et al., 2024). Our critique of runaway escalation is thus specific to the unbounded linear penalty: augmented-Lagrangian, PI-controlled, and resilient updates are all better behaved, yet the resilient method we benchmark still winds up slowly on a hard count (Sec. 5.4). We do not run a dedicated augmented-Lagrangian baseline. Its quadratic penalty growth is structurally close to the ρ ramp of the package we study (both bound the constraint-term gradient after a first violation), so the two are not independent tests of the same fix. The resilient baseline instead represents the modern-controllers family with adaptive constraint tightness. Our two dual-ascent baselines are Fioretto-LDF (Fioretto et al., 2020) and Hounie-RCL (Hounie et al., 2023); the package we study optimizes a bounded penalty and freezes the multiplier at satisfaction. The reset and hinge components we identify (Sec. 3) act on the optimizer state and the constraint direction, not on the multiplier update itself. They are therefore complementary to PI-controlled or resilient dual methods rather than competing (see the graft experiment in Sec. 6).

Output-level and fairness constraints. Closer to our output-level setting, prediction constraints have been enforced during training by primal-dual updates (Nandwani et al., 2019) and by linear label-count constraints for weakly supervised segmentation (Pathak et al., 2015), or guaranteed per prediction by input-dependent parametrizations (Brosowsky et al., 2021) or an output-layer satisfiability solver (Goyal et al., 2024). Márquez-Neila et al. (2017) find hard enforcement feasible but no better than soft penalties, motivating our bounded penalty. Constrained optimization has protected under-represented classes (Sangalli et al., 2021), and group-wise constraints underpin fair classification (Zafar et al., 2017; Agarwal et al., 2018; Kearns et al., 2018; Donini et al., 2018), which our local caps resemble structurally, though a local cap is a *ceiling*, not a fairness floor (Sec. 4).

Rate-constrained learning. The closest classical relatives constrain prediction *rates*: coverage as a dataset constraint (Goh et al., 2016), non-decomposable rate objectives through surrogate bounds (Eban et al., 2017), non-differentiable rate constraints through proxy-Lagrangian games (Cotter et al., 2019), and generalized rate metrics through multi-player schemes (Narasimhan et al., 2019). These bound *expected* rates over a distribution, inductively; ours is a hard count on one specific test set, verified on the argmax predictions and paired with per-group local caps.

Post-hoc correction and resource allocation. Resource-constrained classification allocates a fixed budget over a trained classifier’s outputs at decision time, classically as a small assignment problem, most recently under exactly our local-plus-global structure (Shifman et al., 2025). Related lines fold the budget into learning via adaptive cost-sensitive weights (Shifman et al., 2023) or predict-and-optimize ranking under stochastic capacity (Vanderschueren et al., 2024). Learning-with-rejection (Cortes et al., 2016) instead prices abstention per instance. All steer expected costs rather than a hard count. Decision-time allocation always meets the budget by editing predictions; our post-hoc baselines are a greedy threshold heuristic and a linear-programming clipper (LP-LG), following the local-and-global allocation formulation of Shifman et al. (2025).

Cost-sensitive and imbalanced training. Reweighting or reshaping the loss changes which examples are emphasized: cost-sensitive thresholds (Elkan, 2001), focal loss (Lin et al., 2017), class-balanced weights (Cui et al., 2019), label-distribution-aware margins (Cao et al., 2019), prior-corrected logits (Menon et al., 2021). None of these guarantees the prediction count, which the constraint-trained methods studied here target directly. Whether a well-tuned imbalanced-training recipe followed by clipping would close the macro-F1 gap over vanilla clipping is a question our current experiments do not settle (Sec. 7).

Transductive and set-level constraints. Transductive inference reasons about a fixed unlabeled test set (Vapnik, 1998; Joachims, 1999), where soft set-level targets have long existed: expectation regularization pushes predicted class proportions toward a prior (Mann & McCallum, 2007), and posterior regularization constrains posterior expectations (Ganchev et al., 2010). Our constraint is transductive in this sense but *hard*: an explicit argmax count budget, with per-group caps, enforced during training.

Few methods target both scopes at once. The families surveyed above address either a single global constraint (dual-ascent methods, rate-constrained learning, expectation regularization) or per-group constraints framed as fairness floors (fair classification), but rarely both a global count cap and per-group local caps that must hold at the same time on argmax predictions over a specific test set. Under exactly this local-plus-global structure, the closest recent work is the post-hoc allocator of Shifman et al. (2025). On the constraint-trained side we know of no baseline that handles both scopes jointly, so our benchmark implementations of Fioretto-LDF and Hounie-RCL required adaptation to enforce a global count together with per-group ceilings on argmax counts (Sec. 4).

3 Problem and Method

Problem. Let the (known) test set contain N items and let $p_\theta(y = c \mid x_i)$ be the model’s probability for the constrained class c on item i . A *global* count constraint requires the number of items predicted c to satisfy $\sum_{i=1}^N \mathbf{1}[\hat{y}_i = c] \leq K$. We parameterize the cap as a percentage L of the constrained class’s true test-set count, so L measures how much of the class’s actual demand the quota admits ($L30$ admits at most 30% of the true count). *Local* caps also bound the count within each group g : $\sum_{i \in g} \mathbf{1}[\hat{y}_i = c] \leq K_g$ (the same percentage of the group’s own class count). Because the indicator is not differentiable, training uses the *soft count* $S = \sum_i p_\theta(y = c \mid x_i)$, while satisfaction is verified with the *hard count* from arg max.

Overview of the package. The package we study — for exposition we refer to it as *TraLO* (Transductive Loss Optimization) — combines four ingredients: a saturating per-constraint penalty (below); a multiplier that ratchets while the count is violated and freezes the moment it is met; an Adam-state reset at first satisfaction; and an undershoot hinge that engages once the count falls below the cap. Not all four ingredients carry equal weight. Sec. 6 and App. B–B.2 show that the constrained-class advantage over the strongest dual-ascent baselines rests specifically on the reset and the hinge, that both transfer as portable additions to the dual baselines, and that the penalty shape after which the package is named is neutral by both ablation and graft. We describe the four ingredients below before returning to the empirical story.

The constraint penalty. For a soft excess $E = \max(0, S - K)$ over a cap K , the per-constraint penalty is

$$\ell(E; K, \rho) = \underbrace{\frac{E}{E + K}}_{\text{rational saturation}} + \rho \underbrace{\frac{(E/K)^2}{1 + (E/K)^2}}_{\text{bounded quadratic}}, \quad (1)$$

where both terms saturate as $E \rightarrow \infty$ (the first to 1, the second to ρ). What actually drives training is the penalty’s *gradient* on the count; differentiating Eq. (1) through $E = \max(0, S - K)$ gives, for $S > K$,

$$\frac{\partial \ell}{\partial S} = \frac{K}{(E + K)^2} + \frac{2\rho}{K} \frac{E/K}{(1 + (E/K)^2)^2}. \quad (2)$$

Both the penalty and this gradient are bounded (App. A). The total objective combines cross-entropy with, per cap, the overshoot penalty and an undershoot hinge h ,

$$L_{\text{total}} = L_{\text{CE}} + \lambda_g (\ell_g + h_g) + \lambda_l (\ell_l + h_l), \quad (3)$$

where $\ell_g = \ell(E; K, \rho)$ is the global-cap penalty and $h_g = \beta \max(0, K - S)/K$ its hinge (weight $\beta=0.5$, discussed below): identically zero while $S \geq K$, hence inert throughout the violated phase, it engages only once the soft count falls below the cap, in practice after first satisfaction. ℓ_l and h_l sum the same two terms over the group caps; every cap (the global one and each group cap) carries its own multiplier, so $\lambda_l (\ell_l + h_l)$ abbreviates $\sum_g \lambda_{l,g} (\ell(E_g; K_g, \rho) + \beta \max(0, K_g - S_g)/K_g)$.

A cap’s penalty depends on the parameters only through its scalar soft count S_c , so its gradient always points along the same fixed direction $\nabla_{\theta} S_c$: the multiplier, ρ , and the penalty’s shape scale that push but cannot redirect it, and under gradient clipping even the scale cancels (Pascanu et al., 2013). This is a partial argument only: λ multiplies only the constraint term and thus shifts the cross-entropy-versus-constraint weighting in the combined update, so the multiplier scale is not provably inert; and the shape reshapes $\ell'(S_c)$ at each E rather than rescaling it, so it falls outside the scalar-rescaling argument entirely. A matched probe with λ differing $\sim 280\times$ trains almost identically (App. A), so the near-neutrality is empirical rather than theoretical. So the saturating penalty of Eq. (1) sets the *scale* of the constraint push, not the update it produces: its shape is neutral by ablation and graft (Sec. 6), and the quality comes instead from Adam’s reset and the undershoot hinge, which change the direction.

Multiplier ratchet and freeze. Rather than dual ascent, λ is *ratcheted*: it increases by a fixed step each epoch *only while the hard count violates the cap*, and is *frozen* the moment the count is first satisfied. The bounded penalty means the search ends at a small frozen multiplier rather than an escalating one; multipliers start at 0.05. The quadratic weight ρ ramps linearly (from 5 toward 100) until first satisfaction and then also freezes. Because λ and ρ both stop moving at first satisfaction, the training procedure is really two-phase: while the count is violated it *searches* for a sufficient multiplier by ratcheting λ (the bounded gradient of Eq. (2) keeps that search ending at a small λ), and once the count is met it *settles*.

Stability, and why a verification step is retained. Freezing λ and ρ at first satisfaction turns the moving-target dual problem into a single fixed objective $F(\theta) = L_{\text{CE}} + \lambda_g (\ell_g + h_g) + \lambda_l (\ell_l + h_l)$: the tail of training descends one objective instead of chasing a multiplier that is still moving. Because the penalty has a nonzero slope ($1/K$) at the cap, a finite frozen multiplier suffices to pin the soft count there rather than merely discourage violations, which is why the package reaches the cap natively once cross-entropy saturates (App. A). Feasibility is defined by the *hard* count while the penalty acts on the *soft* count, so a verification step (App. C) closes that small residual gap.

Optimizer reset at satisfaction. At the first epoch the count is met, we reset the Adam optimizer state. This removes the accumulated descent momentum that would otherwise keep pushing the soft count far below the cap (over-satisfying at the cost of constrained-class recall). The reset lets the frozen-multiplier objective settle near the boundary instead of overshooting inward. It is motivated empirically, not derived, and other reset variants (partial reset of only first or only second moments, or decay-based) are not covered by our experiments; the ablation (App. B) isolates the effect of the full reset used here. The reset is applied only at the constraint-satisfaction phase transition, and is distinct from continuous learning-rate restart schedules (Loshchilov & Hutter, 2017), which are orthogonal to the multiplier ratchet.

Undershoot hinge. A satisfied cap can still leave the model under-predicting the constrained class, which costs recall. The hinge term h in Eq. (3) counters this: each cap c contributes $\lambda_c \beta \max(0, K_c - S_c)/K_c$, zero whenever the soft count sits at or above its cap, so it can never re-violate; once the soft count drops below the cap it gently pushes it back up toward (but not over) the cap, restoring constrained-class recall. App. B shows the hinge and the reset together carry the effect.

Training schedule. Training has a warmup phase (cross-entropy only, to a stable initialization that is cached and reused) followed by the constraint phase under Eq. (3). A post-hoc verification step is available

Algorithm 1 The training loop of the package studied here.

```

1: warm up  $\theta$  on  $L_{\text{CE}}$  alone; cache the warmup model
2: for each constraint epoch do
3:   update  $\theta$  on  $L_{\text{CE}} + \lambda_g \ell_g + \lambda_l \ell_l$  (plus the hinge once satisfied)
4:   recount hard predictions on the test inputs
5:   if a cap is violated then
6:     ratchet that cap’s  $\lambda$  up one step; advance  $\rho$ 
7:   else if first time all caps are met then
8:     freeze  $\lambda$  and  $\rho$ ; reset the Adam state
9:   end if
10:  stop after 5 consecutive satisfied epochs
11: end for
12: post-hoc verification (flip borderline predictions) if needed

```

to enforce a hard count if a run has not landed natively; for the reset+hinge package it is essentially never needed (App. C). Algorithm 1 summarizes the loop.

4 Experimental Setup

Datasets. We use three MedMNIST v2 (Yang et al., 2023) classification tasks with their official splits (Figure 1 shows DermMNIST examples): **TissueMNIST** (8 classes, constrained class GE, 7.1% of the test set), **DermMNIST** (7 skin-lesion classes from HAM10000 (Tschandl et al., 2018), constrained class melanoma, 11.1%), and **OctMNIST** (4 retinal-OCT classes, constrained class drusen, 25% of a class-balanced test set). The first two carry the minority class that the screening motivation assumes; OctMNIST does not, and we report it because the cap binds hardest there. All three are 28×28 images upsampled to 224×224 for the ImageNet-pretrained backbones. Local caps use real groups only on DermMNIST (HAM10000 anatomical site: torso/extremity/head-neck); TissueMNIST and OctMNIST lack group metadata and use a fixed synthetic split, a mechanism test rather than a fairness setting (App. E). Caps are set as a percentage L of the constrained class’s true test-set count (Sec. 3); the asymmetric study decouples the two cap types, keeping L for the per-group caps and sweeping the global-cap percentage G independently, so $G < L$ makes the global cap the binding one. That study runs all 20 off-diagonal (L, G) pairs from $\{20, 30, 50, 70, 80\}\%$ on DermMNIST and TissueMNIST (MobileNetV3, all methods, seeds 1–4), re-running four shared $G < L$ pairs on RegNetY-400MF and ViT-B/16 (App. D). The percentage form only standardizes how hard the cap binds across datasets; the method itself receives an exogenous integer K , exactly as a deployed capacity would be given. All headline claims in this paper are scoped to MedMNIST-derived tasks; extending to native-resolution medical or general vision benchmarks is deferred to future work (Sec. 7).

Backbones. The headline grid uses three backbones, MobileNetV3 (Howard et al., 2019), RegNetY-400MF (Radosavovic et al., 2020), and a ViT-B/16 transformer (Dosovitskiy et al., 2021), to check that the findings are not CNN-specific. All backbones use standard ImageNet-style training conventions (He et al., 2016).

Baselines. Two *post-hoc* *clippers*: a greedy threshold **Heuristic** and a linear-programming clipper **LP-LG** (following the local-and-global allocation formulation of Shifman et al. 2025). Two constraint-*trained* dual-ascent methods: **Fioretto-LDF** (Fioretto et al., 2020) and **Hounie-RCL** (Hounie et al., 2023). We also report **TraLO-bounded**, an ablation of our own package (“bounded” names the bare saturating penalty it retains) that strips both load-bearing additions, the optimizer reset and the undershoot hinge, used as a method and as a within-cell control in App. B.

Baseline adaptations. The two trained baselines were built for a different setting, so we re-instantiated each on the transductive count task and ran it inside the same harness as our own package: the same cached warmup, 300-epoch budget and stop rule (five consecutive feasible epochs), gradient clipping, and post-hoc verification step. Two details are specific to the port: each method’s constraint gradient targets the unlabeled

test set, where the count is enforced (the originals target a training or population expectation), and each method’s reported checkpoint is selected on the constraint-excess axis (best feasible, else lowest excess), never on F1, so no baseline is credited with its best-F1 epoch. **Fioretto-LDF** is otherwise unchanged, with no analog of the sample-size rescaling applied to Hounie-RCL below: a linear penalty $\lambda_c S_c$ with per-constraint subgradient ascent $\lambda_c \leftarrow \lambda_c + \eta \max(0, S_c - K_c)$ on the test counts. **Hounie-RCL** keeps its primal/slack/dual updates, with the constraint residual read as $(S_c - K_c)/N$. The one substantive fix in our port is to normalize its primal count-gradient by the sample count so the primal step and the dual update share a scale. Otherwise the primal term is $N \times$ too strong and over-suppresses the capped class. **LP-LG** is post-hoc only: a global-and-local linear program that allocates the capped labels on the warmup model’s test-set softmax, using an identity misclassification cost rather than the general cost matrix of the original two-phase allocator of Shifman et al. (2025).

Frozen recipe. All methods share one protocol, fixed before the final runs: ImageNet-pretrained backbones, 224×224 inputs, batch size 64, no augmentation, Adam with learning rate 10^{-4} during warmup and 5×10^{-6} in the constraint phase; warmup = 50 epochs (cross-entropy only), constraint phase = 300 epochs, seeds 1–4 (all reported, no seed selection). Compute is modest: the constraint phase runs a median 2–7 minutes per run on one GPU for our package and the dual baselines alike, on top of a shared cached warmup (clipping itself costs seconds but pays the quality tax of Sec. 5.3). Per-method constraint steps were fixed a priori, not tuned per cell: dual step 0.005 (Fioretto-LDF), dual and relaxation steps 0.01 (Hounie-RCL) on all three datasets, and ratchet step 0.002 with hinge weight $\beta=0.5$ (TraLO). A step-fairness sweep (App. A) varies the baseline steps over a $10 \times$ range, and a matching sweep varies our own β and λ_{step} (App. B.3): the OctMNIST tight-cap advantage is insensitive to both (the best baseline variant still trails by 0.019; TraLO clears the baseline at every arm). The headline grid is *paper_final*: 3 backbones $\times$ 3 datasets $\times$ 9 cap levels $\times$ 6 methods $\times$ 4 seeds = 1944 runs. Models are implemented in PyTorch with torchvision backbones and MedMNIST v2 loaders, and each run trains on a single commodity NVIDIA GPU, with no multi-GPU or distributed training. Code, the configuration generators, and the full per-run results corpus accompany the submission as the Code and Data Supplement.

Metrics. Our *primary* metrics are quality: **constrained-class F1 (cc-F1)**, the quality of the capped class itself and the quantity the constraint most directly puts at risk, and **macro-F1** for overall quality. As *secondary* deployment diagnostics we also report **native satisfaction** (the fraction of runs whose final model meets the global count with no post-hoc edit) and **flips** (the number of test predictions the post-hoc verification step overwrites to enforce the global and per-group caps; 0 when the run needs no adjustment); we treat these as deployment *properties* rather than headline metrics, since every deployable method meets the hard count one way or another (App. C). Gaps versus baselines are *paired* by seed: because the seeds are matched, we summarize a method’s advantage by its seed-winrate and a paired Wilcoxon test rather than by marginal $\pm$ std bars, which would understate a paired effect when a baseline is seed-unstable but its per-seed gap to TraLO is consistent. Claims spanning several cells are tested on cell means (cells are the independent units; the four seeds recur across cells); within one cell, $n=4$ comparisons are reported as effect sizes and seed counts, since a four-pair Wilcoxon cannot fall below $p=0.125$.

Family-wise multiple-comparisons control. With many cells and multiple axes of comparison, some fraction of significant paired tests are expected under the global null. Rather than apply a blanket Bonferroni correction — very conservative under our paired design, since it would penalize the same four seeds recurring across cells — we report all seed-winrates, effect sizes, and cell-level p -values, so that a Benjamini-Hochberg step-up procedure at any target FDR can be recomputed from the released per-run corpus. As a check, the OctMNIST tight-cap headline (six cells, cell-level sign test $p=0.031$ per load-bearing component) still survives a Benjamini-Hochberg step-up at $q=0.05$ across the twelve cell-by-component comparisons behind Table 2: the six-cell floor $p=0.031$ maps to a BH threshold of $q=0.05 \times 6/12 = 0.025$ for the smallest p -value and $q=0.05 \times 12/12 = 0.05$ for the largest, and both load-bearing components lie under their respective thresholds.

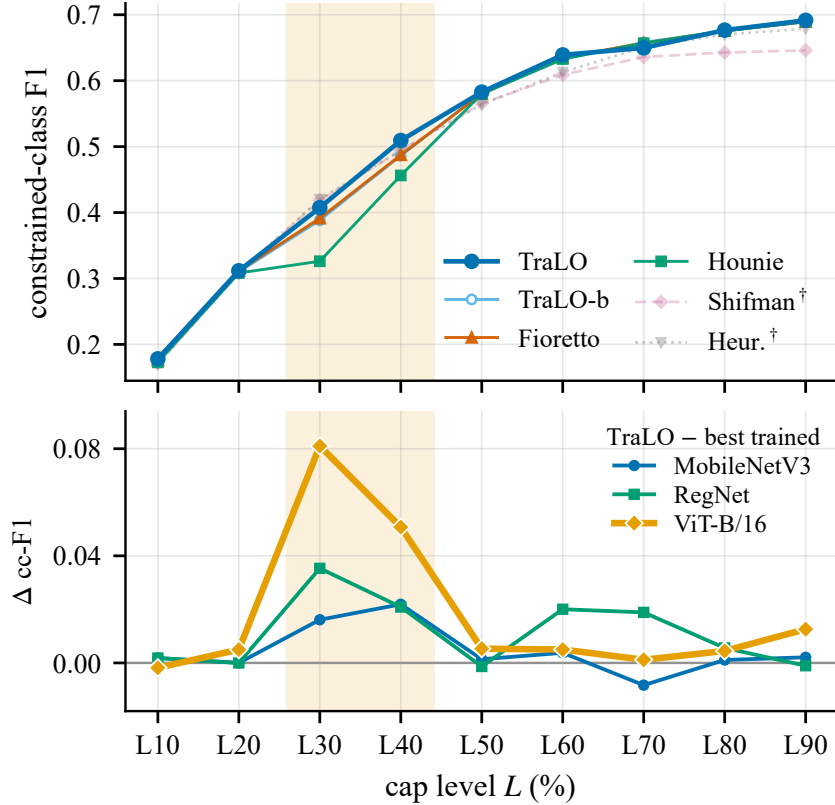


Figure 2: **The OctMNIST tight-cap advantage is backbone-general and largest on the ViT-B/16.** *Top:* constrained-class F1 of all six methods (MobileNetV3, mean over $n=4$ seeds); the curves nearly coincide, so the advantage is invisible at this scale (shaded band = tight-binding regime; [†] faded post-hoc clippers can post higher raw cc-F1 by editing). *Bottom:* TraLO minus the best constraint-trained baseline, paired by seed, per backbone; the same inverted-U appears on all three, largest on the ViT-B/16.

5 Results

5.1 Constrained-class quality vs. the trained baselines

We start with constrained-class F1 (cc-F1), the quality of predictions on the very class the cap restricts; Table 1 shows the full symmetric grid, all three datasets over all three backbones, on both headline metrics (cc-F1 here, macro-F1 in Sec. 5.3). The sharpest result is where the count binds hard: at the OctMNIST tight caps ($L30/L40$, the shaded band of Figure 2) the package leads the best constraint-trained baseline on every backbone, by a mean cc-F1 gap of +0.038 across the six cells (sign test $p=0.031$) and peaking at +0.081 on the ViT-B/16 (per-backbone paired deltas in Table 7). Where the cap is slack the summary is a tie: TraLO is best-or-tied on cc-F1 (within 0.005) against the two trained baselines in 23 of the 27 cells (22 when our own TraLO-bounded ablation variant is also counted as a competitor), and on tissue and derm the differences at equal feasibility are within seed noise (paired Wilcoxon, not significant), a tie that also holds away from the grid, at a tighter ($L40$) and a looser ($L60$) cap on RegNet.

The hard regime: OctMNIST tight caps. The one place the package pulls clearly ahead of the trained baselines is OctMNIST at tight caps, where the cap binds hard but the drusen class is still discriminable. There, TraLO leads the best trained baseline on cc-F1 at every backbone and every seed (4/4 per cell, 24 of 24 seed-level paired comparisons across the six cells), with the margin largest on the ViT-B/16; the six tight-cap deltas are in Table 7. Treating cells as the independent unit, the six cell-mean gaps are all positive;

Table 1: **Quality of the final satisfying predictions on the full symmetric grid:** constrained-class F1 and macro-F1, all three datasets $\times$ all three backbones (mean over seeds 1–4; across-seed stds omitted per cell for width, range .001–.055). **Bold** = best, underline = second-best among the constraint-trained methods; $\dagger$ = post-hoc clippers, for reference; seed-winrates in Supp. Table S6 and Supp. Sec. D. Among trained methods the grid is a tie on Tissue and Derm while TraLO leads every backbone at the OctMNIST tight caps; on macro-F1 TraLO’s paired gap over the best clipper clears +0.005 in 24 of 27 cells, zero losses (grid mean +0.031). Per-dataset paired-gap summaries over the symmetric caps for each backbone (including MobileNetV2) are in Table 6.

Data	Backbone	Cap	Constrained-class F1						Macro-F1					
			Heur. $\dagger$	LP-LG $\dagger$	Fioretto	Hounie	TraLO-b	TraLO	Heur. $\dagger$	LP-LG $\dagger$	Fioretto	Hounie	TraLO-b	TraLO
Tissue	MNetV3	L30	0.225	0.221	0.275	0.275	<u>0.270</u>	0.275	0.347	0.347	0.405	<u>0.406</u>	0.405	0.408
		L50	0.303	0.300	<u>0.352</u>	0.348	<u>0.352</u>	0.354	0.357	0.357	0.421	<u>0.420</u>	<u>0.420</u>	0.417
		L70	0.344	0.326	0.393	0.392	0.404	<u>0.399</u>	0.362	0.360	0.426	<u>0.427</u>	0.429	0.426
	RegNet	L30	0.241	0.239	<u>0.286</u>	0.291	<u>0.286</u>	0.273	0.329	0.328	0.412	0.410	<u>0.411</u>	0.409
		L50	0.309	0.309	0.354	0.356	<u>0.360</u>	0.366	0.337	0.337	0.421	<u>0.422</u>	0.428	<u>0.422</u>
		L70	0.352	0.349	0.407	0.414	<u>0.412</u>	0.409	0.341	0.340	0.430	0.435	<u>0.432</u>	<u>0.432</u>
	ViT-B/16	L30	0.318	0.320	0.327	<u>0.313</u>	0.327	0.327	0.447	0.447	<u>0.465</u>	0.466	<u>0.465</u>	<u>0.465</u>
		L50	0.411	0.414	<u>0.420</u>	0.422	0.422	0.418	0.458	0.459	0.481	<u>0.482</u>	<u>0.482</u>	0.483
		L70	0.473	0.469	0.466	0.459	<u>0.464</u>	0.462	0.466	0.466	<u>0.487</u>	0.489	<u>0.487</u>	<u>0.487</u>
Derm	MNetV3	L30	0.419	0.417	<u>0.417</u>	0.412	0.419	0.419	0.722	0.721	0.723	0.728	<u>0.727</u>	0.726
		L50	0.557	0.557	<u>0.561</u>	0.560	<u>0.561</u>	0.564	0.747	0.747	<u>0.760</u>	0.761	0.759	0.761
		L70	0.633	0.633	<u>0.645</u>	0.641	<u>0.645</u>	0.648	0.764	0.764	0.784	0.784	0.784	<u>0.781</u>
	RegNet	L30	0.422	0.422	0.419	0.417	<u>0.422</u>	0.424	0.703	0.703	0.705	0.703	<u>0.706</u>	0.708
		L50	0.549	0.548	0.540	0.545	0.539	<u>0.543</u>	0.726	0.726	0.735	0.735	0.735	<u>0.730</u>
		L70	0.617	0.612	<u>0.621</u>	0.620	0.623	0.612	0.740	0.740	0.753	<u>0.752</u>	0.753	0.749
	ViT-B/16	L30	0.371	0.369	0.386	<u>0.384</u>	0.386	0.386	0.646	0.646	0.674	0.674	0.677	<u>0.676</u>
		L50	0.487	0.488	<u>0.507</u>	<u>0.507</u>	0.510	<u>0.507</u>	0.664	0.664	0.700	<u>0.698</u>	0.700	0.700
		L70	0.550	0.542	<u>0.575</u>	0.571	0.578	<u>0.575</u>	0.674	0.672	0.705	0.724	<u>0.707</u>	0.702
OctMNIST	MNetV3	L30	0.420	0.418	<u>0.391</u>	0.326	0.388	0.407	0.666	0.665	<u>0.701</u>	0.678	0.700	0.702
		L50	0.564	0.565	0.580	0.580	<u>0.581</u>	0.583	0.712	0.713	<u>0.762</u>	<u>0.762</u>	<u>0.762</u>	0.763
		L70	0.651	0.636	<u>0.657</u>	<u>0.657</u>	0.658	0.650	0.741	0.737	0.783	0.785	<u>0.784</u>	0.781
	RegNet	L30	0.402	0.400	<u>0.372</u>	0.299	0.369	0.407	0.694	0.693	<u>0.688</u>	0.665	0.686	0.700
		L50	0.536	0.535	<u>0.545</u>	0.541	0.547	0.544	0.734	0.734	<u>0.743</u>	0.742	<u>0.743</u>	0.745
		L70	0.621	0.577	0.624	0.626	<u>0.629</u>	0.645	0.759	0.746	0.768	0.768	<u>0.770</u>	0.777
	ViT-B/16	L30	0.440	0.437	0.335	0.332	<u>0.352</u>	0.422	0.718	0.717	0.692	0.691	<u>0.697</u>	0.722
		L50	0.600	0.600	0.592	0.588	<u>0.595</u>	0.597	0.771	0.771	<u>0.779</u>	0.776	<u>0.779</u>	0.781
		L70	0.671	0.653	<u>0.671</u>	0.670	<u>0.671</u>	0.672	0.795	0.790	0.805	<u>0.804</u>	0.805	0.805

the t -test on the same six cell means gives $p=0.013$, and the same cells improve macro-F1 at five of the six (the sixth a +0.001 wash; mean +0.012). The advantage is recall at the cap, not precision: TraLO matches the best trained baseline on constrained-class precision (0.90 vs 0.91) and beats it on recall (0.31 vs 0.27), the quantity the undershoot hinge restores. The duals lose that recall from below: penalized only above the cap, they overshoot it downward and finish 11–36% under the budget at $L30$ (cell-mean counts 48–67 of $K=75$, vs 73–74 for TraLO), so recall falls with the unspent budget while precision holds. A tight cap holds recall near its ceiling K/N at high precision, and training recovers the leftover headroom: a property of the cap, not the 28×28 resolution. Figure 2 plots the sweep: the six methods nearly coincide (top), while the per-backbone gap (bottom) reveals the advantage and its inverted-U. Where a post-hoc clipper posts higher raw cc-F1 (three of the six band cells, by up to +0.018), the edge is bought by editing: at OctMNIST $L30$ on MobileNetV3, clipping keeps the base model’s 75 most confident drusen calls and rewrites the ~ 110 evicted predictions (11% of the batch) to each sample’s runner-up class, so the bill lands off-class and TraLO leads that same cell 0.702 to 0.666 on macro-F1 (App. C).

Robustness of the OctMNIST advantage. Three checks. (i) *Backbone-general*: leave-one-backbone-out cross-validation (define the winning band on the two training backbones, test on the held-out one;

script in the released corpus) holds on every held-out backbone: pooled held-out gap +0.033 at a 93% seed-winrate, and 100% on the tight caps themselves. (ii) *Re-derivable without the gap*: a rule that never sees the TraLO-vs-baseline gap flags a cell when the cap binds (the clipper is forced into ≥ 65 flips) and the class is still discriminable (mid-range cc-F1: recall headroom left at high precision). Every flagged cell is a TraLO advantage (win-rate 100%), vs 0–40% on unflagged cells (0% over-tight, rising to 40% at loose caps); the discriminability test reads test labels, so this re-derives the regime for analysis, not a deployment-time selector. (iii) *Inverted-U, not monotone*: the effect is an inverted-U in tightness, not “tighter is always better,” and it is OctMNIST-specific. This is the one *backbone-general* cc-F1 advantage; the full regime map follows (Sec. 5.2).

5.2 Wins, ties, and losses across the full grid

Across the full corpus the observed pattern is: **TraLO tends to win where the count binds hard and the constrained class stays discriminable, and to tie where either condition fails.** Binding alone is not enough: at $L30$ the warmup model over-demands the cap by 2.4–4.8 $\times$ on tissue and derm, more than OctMNIST’s 2.0–2.5 $\times$, yet only OctMNIST clears the win bar. We score the *paper_final* grid on the three backbones plus dedicated MobileNetV3 asymmetric sweeps, partitioned into five constraint-geometry regimes (tight, middle, and loose symmetric caps: $L=G \leq 40, =50, \geq 60$; asymmetric caps with the global side, $G < L$, or the group side, $G > L$, tighter), every regime reported (per-cell breakdown in App. D). A regime counts as a *win* only when the mean paired cc-F1 gap and at least half its cells clear +0.005, and a *loss* under the mirror-image rule; no regime mean falls below -0.005 . The +0.005 effect-size threshold was chosen from inspection of the ablation and graft magnitudes in Sec. 6 rather than pre-registered; the OctMNIST tight-cap regime survives generously wider thresholds (paired cell-mean gaps range +0.016 to +0.081; Table 7), so its classification does not turn on the threshold choice, while the DermMNIST asymmetric result sits closer to the threshold and is reported as *preliminary* for that reason as well as for its MobileNetV3-only backbone coverage. Only two regimes clear the win bar: OctMNIST tight-symmetric caps (the backbone-general advantage of Sec. 5.1, largest on ViT-B/16) and the global-tighter asymmetric ($G < L$) caps of DermMNIST (+0.8pp over MobileNetV3’s full asymmetric grid, reproduced on RegNet and tied on ViT over the four shared cap pairs; *preliminary*, as only MobileNetV3 has the full grid). Every other regime, including all symmetric caps, is a statistical tie (symmetric tissue+derm mean cc-F1 gap -0.0013 , seed winrate 29%, $n=216$), and none is a loss. Against the post-hoc clippers macro-F1 is higher for TraLO on every regime (Sec. 5.3). All of these statements are scoped to the three MedMNIST-derived datasets we ran.

5.3 Quality vs. post-hoc clipping

The other baseline family is the field default: train unconstrained, clip the top-scoring predictions down to the cap. Both end up meeting the count; the question is what quality that costs. Clipping meets the cap by overwriting predictions (often dozens per run), an accuracy tax that the constraint-trained methods — landing on the count natively — avoid. Across the full 1944-run grid (run-level, paired by seed, against the best post-hoc clipper) this is a macro-F1 gain for TraLO of +0.053 (Tissue), +0.016 (Derm) and +0.021 (OctMNIST), positive in every regime and, by cell-mean paired gap, in 77 of the 81 symmetric cells (per cell at the displayed caps in the macro-F1 block of Table 1; App. D has all 81). The four exceptions, two of them ties within 0.001, sit at the very tightest $L10/L20$ caps, where the cap pins every method’s constrained-class recall at the same ceiling, so the margin is decided off-class and the clipper’s frozen base model keeps its unconstrained accuracy there. The gain comes with constrained-class F1 moving the same way in aggregate. The margin over clipping is a property of training under the constraint rather than of TraLO alone: the dual baselines post nearly identical macro-F1 margins (the lone exception a tie at OctMNIST $\times$ ViT), so we claim it for the constraint-trained family; constraint-time training also adapts to the test batch, a confound we do not separate from avoided editing in the current experiments (Sec. 7). What is TraLO’s own is reaching that quality with the fastest and most reliable native convergence (Sec. 5.4) and the fewest prediction edits at binding caps. The tax is steepest where the cap binds: at the tight $L30$ cap, clipping costs up to 0.061 macro-F1 (Tissue; App. C). How TraLO avoids the tax (near-perfect native satisfaction, an order of magnitude fewer edits) is a deployment property reported in App. C: what separates the methods is the quality each preserves while meeting the count. In deployment terms: train under the constraint when the

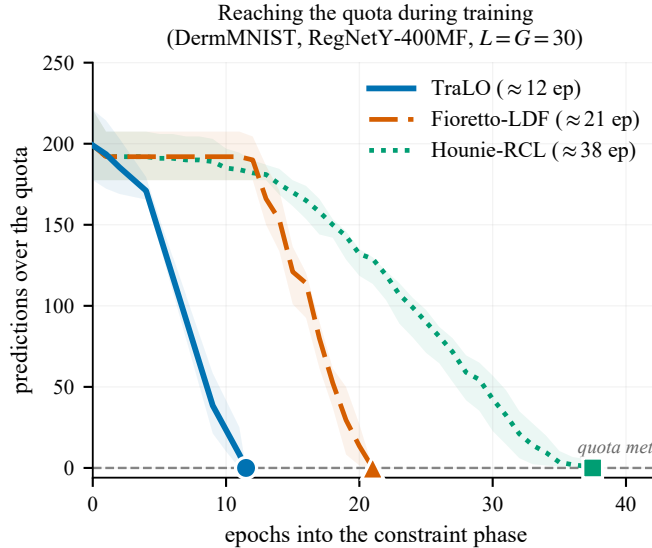


Figure 3: **TraLO reaches the count fast and holds it.** Best-so-far constraint excess (lowest predictions over the cap so far) over the constraint phase; DermMNIST (RegNetY-400MF), tight cap $L=G=30$. Median over $n=4$ seeds; band: interquartile range; markers at each method’s median epoch-to-feasibility. All three start about 200 over the cap; on this cell TraLO’s ratchet reaches the cap first, the zero-start duals later. Giving the baselines TraLO’s nonzero start narrows but does not close this (App. B.4).

full label vector ships, and clip only when the capped shortlist is the sole deliverable, a shortlist assembled by rewriting 5–11% of the batch at binding caps.

5.4 Convergence

These advantages rest on TraLO reaching the count during training. Figure 3 tracks how far each method’s prediction count sits above the cap over the constraint phase, on the cell where the three methods separate most cleanly: DermMNIST on RegNet, whose cross-entropy saturates slowly enough that the count and the task objective openly compete. The ratcheted multiplier acts from the first constraint epoch and reaches the cap well before either zero-start dual baseline (per-method epochs in the caption). The gap holds grid-wide: native feasibility is 100% for TraLO and Hounie-RCL but only 87% for Fioretto-LDF, whose unbounded multiplier stalls short of the cap on the rest. Across DermMNIST and TissueMNIST on the two convolutional backbones, over the 18 symmetric-cap cells we log, Hounie-RCL is the slowest of the three in every one and TraLO is never the slowest: TraLO reaches the count first or within one epoch in 15 of the 18 (strictly first in 11), at median epochs-to-feasibility of 17, 20, and 49.5 for TraLO, Fioretto-LDF, and Hounie-RCL. Fioretto-LDF is competitive when it converges but stalls on the runs it does not, so TraLO is the only method that reaches the count both fast and on every run, which lets it avoid the editing that costs clipping its quality. The per-cell census accompanies the released corpus.

6 What Carries the Win: Ablation and Graft

The previous section reports TraLO’s advantages over the constraint-trained and post-hoc-clipping baselines. This section asks the internal question: *which of the four ingredients in the package produces those advantages, and are they specific to TraLO or transferable?* Two converging experiments answer it: a within-cell ablation that removes each ingredient from TraLO and re-runs the OctMNIST tight-cap cells (App. B), and a cross-method *graft* that adds the same two ingredients to each dual baseline and re-runs the same cells (App. B.1). A third experiment (App. B.2) substitutes a linear penalty for Eq. (1) to isolate the contribution of the penalty shape itself.

Ablation isolates the reset and the hinge. On the six OctMNIST tight-cap cells where TraLO leads, we disable one component at a time and measure the paired change in constrained-class F1 ($n=24$ per component; Table 2). Two components are load-bearing. Removing the Adam-state reset at satisfaction costs $+0.079$ cc-F1, the single largest effect, helping on all 24 paired runs and hence in all six cells (cell-level sign test $p=0.031$). Removing the undershoot hinge (the TraLO-bounded control, already in the main grid) costs $+0.036$ cc-F1 (positive on all 24 paired runs; cell-level sign test $p=0.031$; $0.019\text{--}0.070$ across cells). The other components are cc-F1-neutral on these already-satisfied cells: the ρ schedule and the lambda freeze move it by under 0.001 (n.s.).

Graft transfers the win to the dual baselines. The converse test adds those two ingredients to the dual baselines. We graft exactly the optimizer reset and the undershoot hinge onto Fioretto-LDF and Hounie-RCL (each using the host’s own multipliers and penalty) and re-run the six tight-cap cells: 144 runs in all, including re-run controls for TraLO and both hosts under identical warmup caches, the frozen recipe, and one GPU model. The re-run controls reproduce the main-grid values to ± 0.0013 , so the arms are compared on equal footing. The graft lifts Fioretto-LDF in six of six cells ($+0.019$ to $+0.073$ cc-F1) and Hounie-RCL in five of six ($+0.035$ to $+0.103$; the one exception, *L40* on RegNet, drops by 0.028), recovering a median 86% of TraLO’s margin over the Fioretto host and 92% over the Hounie host (medians over the six cells, the loss cell included). Against each grafted host separately TraLO stays best-or-tied (8 wins, 4 ties, 0 losses under the main-text win rule). Seed-by-seed against the stronger of the two grafted hosts, though, TraLO’s mean edge is only $+0.0015$ and it trails in two of the six cells: *once the recipe is grafted the three methods are statistically interchangeable*, with what remains of TraLO’s lead concentrated on the ViT-B/16 ($+0.015/+0.017$ over the grafted Fioretto-LDF at the two caps). This is the paper’s central mechanistic finding: the two portable ingredients, not the host, carry the constrained-class advantage.

Penalty shape is neutral. A third arm swaps Eq. (1)’s saturating penalty for a linear one at the same ratcheted multiplier, holding the ratchet, the freeze, the reset, and the hinge fixed — so the only change is the shape after which the package is named. The arm is genuinely different (its logged constraint loss runs about $3\times$ smaller throughout); its predictions are not. cc-F1 moves by -0.001 (the bounded shape wins 9 of 24 runs and loses 9, with 6 exact ties; sign-test $p=1.0$) and median time-to-feasibility by two epochs. In 6 of the 24 runs the two arms land on *identical* cc-F1: changing the penalty shape changed no prediction at all, consistent with the fixed-direction argument of Sec. 3. We report this as a negative result about our own formulation: Eq. (1) earns its place by keeping the frozen multiplier small and readable, not by earning the constrained-class gains, which belong to the reset and the hinge.

Anti-windup as a mechanistically-limited control. As a further control, Fioretto-LDF with dual restarts (Gallego-Posada et al., 2022) deploys predictions *identical* to plain Fioretto-LDF in every cell, because the deployed checkpoint is selected on the constraint-excess axis before first feasibility and the restart machinery only acts after feasibility. The control therefore cannot in principle change the deployed model. What it does show is that the linear-penalty comparison in the main grid is not a strawman created by omission of a standard post-feasibility fix; it does not rule out that a stronger anti-windup scheme acting before feasibility could close the gap.

7 Limitations

We foreground the limitations because several of them are load-bearing for how the reader should generalize.

Dataset scope. All three datasets are drawn from MedMNIST v2 and are 28×28 images upsampled to 224×224 for the ImageNet-pretrained backbones. Every headline claim in this paper is scoped to that setting. We do not know how the reset and hinge transfer to native-resolution medical imaging (say ISIC-2019 at full resolution), to non-medical long-tailed benchmarks (ImageNet-LT), or to text/graph settings. The regime map itself may re-derive at other scales — the mechanistic story (dual ascent overshoots downward, hinge restores recall) is not resolution-specific — but this is a hypothesis, not an evidenced claim.

The advantage over trained baselines is regime-specific. Only two regimes clear the win bar: OctMNIST tight symmetric caps (the backbone-general result) and the DermMNIST global-tighter asymmetric caps (preliminary, MobileNetV3-only for the full grid). Every other regime is a statistical tie. A reader who cares only about tight-cap regimes on discriminable classes will find the advantage useful; a reader in the tie region has no reason to prefer TraLO to a grafted dual baseline on constrained-class quality.

Motivation–result mismatch on OctMNIST. The screening motivation in Sec. 1 imagines rare, high-stakes classes. OctMNIST’s drusen class is neither: OctMNIST’s test split is class-balanced by construction (250 items per class), and drusen is a relatively benign OCT finding compared with CNV or DME. Readers should not take the OctMNIST result as evidence about rare-class clinical screening; a clinical deployment would more plausibly cap the urgent classes than drusen. The OctMNIST cells earn a place in this paper because the cap binds hardest there and the class stays discriminable, not because the story matches deployment.

Confound between constraint-time training and test-batch adaptation. The macro-F1 gain over post-hoc clipping is claimed for the constraint-trained family in aggregate. But constraint-time training on the test batch (transductive fitting) is a second thing our methods do that vanilla clipping does not, and the current experiments do not separate the two: some of the +1.6–5.3 pp macro-F1 gain plausibly comes from test-batch adaptation rather than from avoided editing per se. An inductive variant (constraint on train-set count only) would separate them; we did not run it.

Missing baselines from imbalanced learning. We compare against constraint-trained dual ascent and post-hoc clipping, but not against focal loss (Lin et al., 2017), class-balanced weighting (Cui et al., 2019), or logit adjustment (Menon et al., 2021) followed by clipping. If any of these closes the macro-F1 gap over vanilla clipping, the “constraint-trained family beats clipping” claim tightens to “constraint-trained beats *vanilla* clipping.” The comparison is a natural next experiment.

The reset is ad hoc; other reset variants are not covered. The optimizer reset at first satisfaction is a full Adam-state reset (first and second moment estimates both zeroed). Partial resets — only m , only v , decay-based rather than binary — are plausible alternatives; our experiments do not cover them. The evidence identifies that *some* reset at satisfaction is load-bearing, not that this specific reset is optimal.

Small n per cell. Each cell has $n=4$ seeds. Within-cell paired Wilcoxon tests cannot fall below $p=0.125$, so we report within-cell effects as effect sizes and seed counts, and reserve p -values for cell-mean statistics ($n=6$ cells at the tight-cap regime). Studies powered to detect small per-cell effects would need more seeds.

Transductive-only scope. The formulation is transductive: it enforces a hard count on a known evaluation batch, scored in one shot. Online/streaming settings where items arrive one at a time and the count budget must be spent over time are outside the scope of this work. A natural extension would replace the ratcheted global multiplier with a running budget-consumption controller and the transductive count with an expected residual on the remaining stream, but we do not evaluate this here.

Naming caveat. The package studied here is named for its saturating penalty shape (Eq. (1)), yet the ablation and graft show the shape is neutral to constrained-class quality. Readers looking to reuse the findings should carry the reset and the hinge, not the specific penalty of Eq. (1): it is a scale-setter, not a quality-setter.

8 Discussion and Conclusion

Count caps are a real, common deployment constraint whose two standard answers each cost something: post-hoc clipping pays an accuracy tax for editing predictions, and dual-ascent training chases the cap with an escalating multiplier and, in its linear-penalty form, can stall short of it. A third answer — constrained training with a saturating penalty, a multiplier frozen at satisfaction, an optimizer reset, and an undershoot

hinge — matches the strong constraint-trained baselines across the grid and beats them on the constrained class where the cap binds hard and the class stays discriminable (OctMNIST tight caps; preliminarily, DermMNIST’s global-tighter asymmetric caps), and improves macro-F1 over post-hoc clipping while landing on the count natively.

Two findings of this paper are worth stating separately from the package. First, the constrained-class advantage over the strongest dual-ascent baselines rests on *two* specific ingredients — an Adam-state reset at first feasibility and an undershoot hinge — that transfer to the dual baselines and close most of the gap, while the saturating penalty after which the package is named proves neutral by both ablation and graft. Second, the regime map shows that hard-count constrained training pays back over clipping essentially everywhere on our grid, while beating the trained baselines only in a narrow, identifiable regime; the map is what a practitioner needs, not the brand of any one package. We release the full corpus so both findings — and every tie and negative result behind them — are independently verifiable at the granularity we report them.

Broader Impact Statement

This paper studies methods that enforce hard prediction quotas in classification systems. Such quotas often arise in screening, audit, or triage pipelines where downstream capacity (specialist time, investigator attention) is limited. There are two potential harms worth flagging. First, a hard cap on flagging a class is a *ceiling* on how many items are escalated, not a floor on how well the population is served; misapplied as a fairness intervention, per-group caps can lock in under-diagnosis of groups where the cap is set from a biased prevalence estimate. Section 2 and App. E distinguish the ceiling reading from a fairness floor, but we cannot control how downstream users set K_g . Second, we evaluate on MedMNIST-derived tasks (28×28 upsampled); the paper does not license clinical deployment on the datasets we use, and readers should not read the OctMNIST result as evidence about rare-class clinical screening (Sec. 7). Beyond these two, the constrained-training recipe has no capabilities beyond ordinary supervised classification and no new dual-use concerns that we can identify.

Reproducibility Statement

All headline claims in this paper are backed by the released code and per-run results corpus. Specifically: the training and evaluation code, the configuration generators, and the raw per-cell/per-seed outputs for all 1944 runs of the headline grid (plus the ablation, graft, anti-windup, penalty-shape, hyperparameter-sweep, initialization, and cross-backbone controls behind Secs. 5.1–6 and Apps. A–E) accompany this submission at <https://anonymous.4open.science/r/tralo-anonymous> (anonymized during review; will be de-anonymized on acceptance). All methods share the same frozen recipe (Sec. 4), fixed before the final runs; per-method constraint step sizes were fixed a priori, not tuned per cell, and are documented alongside the code. The four seeds (1–4) are all reported, with no seed selection. Cell-level statistics (paired Wilcoxon, sign tests, seed-winrates) are recomputable from the released per-run corpus using the analysis scripts included. Compute for the full grid is modest (median 2–7 minutes per constraint-phase run on one commodity GPU), which we hope keeps external replication practical.

References

- Alekh Agarwal, Alina Beygelzimer, Miroslav Dudík, John Langford, and Hanna Wallach. A reductions approach to fair classification. In *International conference on machine learning*, pp. 60–69. PMLR, 2018.
- Dimitri P Bertsekas. *Constrained optimization and Lagrange multiplier methods*. Academic Press, 1982.
- Mathis Brosowsky, Florian Keck, Olaf Dünkler, and Marius Zöllner. Sample-specific output constraints for neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pp. 6812–6821, 2021.
- Kaidi Cao, Colin Wei, Adrien Gaidon, Nikos Arechiga, and Tengyu Ma. Learning imbalanced datasets with label-distribution-aware margin loss. *Advances in neural information processing systems*, 32, 2019.

- Luiz Chamon and Alejandro Ribeiro. Probably approximately correct constrained learning. *Advances in Neural Information Processing Systems*, 33:16722–16735, 2020.
- Luiz FO Chamon, Santiago Paternain, Miguel Calvo-Fullana, and Alejandro Ribeiro. Constrained learning with non-convex losses. *IEEE Transactions on Information Theory*, 69(3):1739–1760, 2023.
- Corinna Cortes, Giulia DeSalvo, and Mehryar Mohri. Learning with rejection. In *International conference on algorithmic learning theory*, pp. 67–82. Springer, 2016.
- Andrew Cotter, Heinrich Jiang, Maya Gupta, Serena Wang, Taman Narayan, Seungil You, and Karthik Sridharan. Optimization with non-differentiable constraints with applications to fairness, recall, churn, and other goals. *Journal of Machine Learning Research*, 20(172):1–59, 2019.
- Yin Cui, Menglin Jia, Tsung-Yi Lin, Yang Song, and Serge Belongie. Class-balanced loss based on effective number of samples. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 9268–9277, 2019.
- Michele Donini, Luca Oneto, Shai Ben-David, John Shawe-Taylor, and Massimiliano Pontil. Empirical risk minimization under fairness constraints. *Advances in neural information processing systems*, 31, 2018.
- Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations (ICLR)*, 2021.
- Elad Eban, Mariano Schain, Alan Mackey, Ariel Gordon, Ryan Rifkin, and Gal Elidan. Scalable learning of non-decomposable objectives. In *Artificial intelligence and statistics*, pp. 832–840. PMLR, 2017.
- Juan Elenter, Luiz F. O. Chamon, and Alejandro Ribeiro. Near-optimal solutions of constrained learning problems. In *International Conference on Learning Representations (ICLR)*, 2024.
- Charles Elkan. The foundations of cost-sensitive learning. In *International joint conference on artificial intelligence*, volume 17, pp. 973–978. Lawrence Erlbaum Associates Ltd, 2001.
- Ferdinando Fioretto, Pascal Van Hentenryck, Terrence WK Mak, Cuong Tran, Federico Baldo, and Michele Lombardi. Lagrangian duality for constrained deep learning. In *Joint European conference on machine learning and knowledge discovery in databases*, pp. 118–135. Springer, 2020.
- Jose Gallego-Posada, Juan Ramirez, Akram Erraqabi, Yoshua Bengio, and Simon Lacoste-Julien. Controlled sparsity via constrained optimization or: How i learned to stop tuning penalties and love constraints. *Advances in Neural Information Processing Systems*, 35:1253–1266, 2022.
- Jose Gallego-Posada, Juan Ramirez, Meraj Hashemizadeh, and Simon Lacoste-Julien. Cooper: A library for constrained optimization in deep learning. *arXiv preprint arXiv:2504.01212*, 2024.
- Kuzman Ganchev, João Graça, Jennifer Gillenwater, and Ben Taskar. Posterior regularization for structured latent variable models. *Journal of Machine Learning Research*, 11:2001–2049, 2010.
- Gabriel Goh, Andrew Cotter, Maya Gupta, and Michael P Friedlander. Satisfying real-world goals with dataset constraints. *Advances in neural information processing systems*, 29, 2016.
- Kshitij Goyal, Sebastijan Dumancic, and Hendrik Blockeel. Deepsade: Learning neural networks that guarantee domain constraint satisfaction. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pp. 12199–12207, 2024.
- Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- Ignacio Hounie, Alejandro Ribeiro, and Luiz FO Chamon. Resilient constrained learning. *Advances in Neural Information Processing Systems*, 36:71767–71798, 2023.

- Andrew Howard, Mark Sandler, Grace Chu, Liang-Chieh Chen, Bo Chen, Mingxing Tan, Weijun Wang, Yukun Zhu, Ruoming Pang, Vijay Vasudevan, et al. Searching for mobilenetv3. In *Proceedings of the IEEE/CVF international conference on computer vision*, pp. 1314–1324, 2019.
- Thorsten Joachims. Transductive inference for text classification using support vector machines. In *Proceedings of the 16th International Conference on Machine Learning (ICML)*, pp. 200–209, 1999.
- Michael Kearns, Seth Neel, Aaron Roth, and Zhiwei Steven Wu. Preventing fairness gerrymandering: Auditing and learning for subgroup fairness. In *International conference on machine learning*, pp. 2564–2572. PMLR, 2018.
- Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- James Kotary, Ferdinando Fioretto, Pascal Van Hentenryck, and Bryan Wilder. End-to-end constrained optimization learning: A survey. In *International Joint Conference on Artificial Intelligence (IJCAI) — Survey Track*, 2021.
- Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE international conference on computer vision*, pp. 2999–3007, 2017.
- Ilya Loshchilov and Frank Hutter. SGDR: Stochastic gradient descent with warm restarts. In *International Conference on Learning Representations (ICLR)*, 2017.
- Gideon S Mann and Andrew McCallum. Simple, robust, scalable semi-supervised learning via expectation regularization. In *Proceedings of the 24th international conference on Machine learning*, pp. 593–600, 2007.
- Pablo Márquez-Neila, Mathieu Salzmann, and Pascal Fua. Imposing hard constraints on deep networks: Promises and limitations. *arXiv preprint arXiv:1706.02025*, 2017.
- Aditya Krishna Menon, Sadeep Jayasumana, Ankit Singh Rawat, Himanshu Jain, Andreas Veit, and Sanjiv Kumar. Long-tail learning via logit adjustment. In *International Conference on Learning Representations (ICLR)*, 2021.
- Yatin Nandwani, Abhishek Pathak, Mausam, and Parag Singla. A primal dual formulation for deep learning with constraints. *Advances in neural information processing systems*, 32, 2019.
- Harikrishna Narasimhan, Andrew Cotter, and Maya Gupta. Optimizing generalized rate metrics with three players. *Advances in neural information processing systems*, 32, 2019.
- Jorge Nocedal and Stephen J Wright. *Numerical Optimization*. Springer, 2nd edition, 2006.
- Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. On the difficulty of training recurrent neural networks. In *International Conference on Machine Learning (ICML)*, pp. 1310–1318, 2013.
- Deepak Pathak, Philipp Krahenbuhl, and Trevor Darrell. Constrained convolutional neural networks for weakly supervised segmentation. In *Proceedings of the IEEE international conference on computer vision*, pp. 1796–1804, 2015.
- John C. Platt and Alan H. Barr. Constrained differential optimization. In *Advances in Neural Information Processing Systems (NIPS)*, 1988.
- Ilija Radosavovic, Raj Prateek Kosaraju, Ross Girshick, Kaiming He, and Piotr Dollár. Designing network design spaces. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pp. 10425–10433, 2020.
- Sara Sangalli, Ertunc Erdil, Andreas Hötter, Olivio Donati, and Ender Konukoglu. Constrained optimization to train neural networks on critical and under-represented classes. *Advances in neural information processing systems*, 34:25400–25411, 2021.

- Danit Abukasis Shifman, Izack Cohen, Kejun Huang, Xiaochen Xian, and Gonen Singer. An adaptive machine learning algorithm for the resource-constrained classification problem. *Engineering Applications of Artificial Intelligence*, 119:105741, 2023.
- Danit Abukasis Shifman, Itay Margolin, Chen Halfi, and Gonen Singer. Classification tasks with local and global resource allocation constraints. *IFAC-PapersOnLine*, 59(1):61–66, 2025.
- Motahareh Sohrabi, Juan Ramirez, Tianyue H. Zhang, Simon Lacoste-Julien, and Jose Gallego-Posada. On PI controllers for updating Lagrange multipliers in constrained optimization. In *International Conference on Machine Learning (ICML)*, 2024.
- Adam Stooke, Joshua Achiam, and Pieter Abbeel. Responsive safety in reinforcement learning by pid lagrangian methods. In *International conference on machine learning*, pp. 9133–9143. PMLR, 2020.
- Philipp Tschandl, Cliff Rosendahl, and Harald Kittler. The ham10000 dataset, a large collection of multi-source dermatoscopic images of common pigmented skin lesions. *Scientific data*, 5(1):180161, 2018.
- Toon Vanderschueren, Bart Baesens, Tim Verdonck, and Wouter Verbeke. A new perspective on classification: Optimally allocating limited resources to uncertain tasks. *Decision support systems*, 179:114151, 2024.
- Vladimir N. Vapnik. *Statistical Learning Theory*. Wiley-Interscience, New York, 1998.
- Jiancheng Yang, Rui Shi, Donglai Wei, Zequan Liu, Lin Zhao, Bilian Ke, Hanspeter Pfister, and Bingbing Ni. Medmnist v2-a large-scale lightweight benchmark for 2d and 3d biomedical image classification. *Scientific data*, 10(1):41, 2023.
- Muhammad Bilal Zafar, Isabel Valera, Manuel Gomez-Rodriguez, and Krishna P. Gummadi. Fairness constraints: Mechanisms for fair classification. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54, pp. 962–970, 2017.

A Mechanism: Bounded vs. Unbounded Dual Behavior

This appendix supports the convergence and mechanism story of Sec. 5.4 in two ways: first, it establishes analytically that the saturating penalty and its gradient are bounded and that a finite frozen multiplier suffices to pin the soft count at the cap; second, it reports a one-epoch-warmup probe that isolates the constraint-descent dynamics from the cross-entropy saturation of the paper grid, exposing where dual ascent applies force it cannot use.

Bounded penalty and gradient. Write $u = E/K \geq 0$. In Eq. (1) the first term $E/(E + K) = u/(1 + u)$ increases from 0 to 1 and the second $\rho u^2/(1 + u^2)$ increases from 0 to ρ , so $0 \leq \ell < 1 + \rho$. Differentiating through $E = \max(0, S - K)$ gives Eq. (2) for $S > K$. There the first summand $K/(E + K)^2 = \frac{1}{K}(1 + u)^{-2}$ is at most $1/K$ (at $u=0$) and decreasing; the second, $\frac{2\rho}{K} u/(1 + u^2)^2$, is maximized at $u = 1/\sqrt{3}$ with value $\frac{9\rho}{8\sqrt{3}K} < \rho/K$ and tends to 0 as $u \rightarrow \infty$. Their sum therefore lies in $[0, (1 + \rho)/K]$ and vanishes as $E \rightarrow \infty$. Scaling ℓ and $\partial\ell/\partial S$ by a frozen multiplier λ gives the bounds $\lambda(1 + \rho)$ and $\lambda(1 + \rho)/K$, neither depending on E .

Rank-one gradient direction. $E_c = \max(0, S_c - K_c)$ is a function of the scalar S_c , and ℓ is a function of E_c , so $\lambda_c \ell(E_c)$ is the composition $\theta \mapsto S_c(\theta) \mapsto \lambda_c \ell$. The chain rule gives $\nabla_\theta[\lambda_c \ell] = \lambda_c \ell'(E_c) (\partial E_c / \partial S_c) \nabla_\theta S_c = \lambda_c (\partial \ell / \partial S_c) \nabla_\theta S_c$, using $\partial E_c / \partial S_c = \mathbf{1}[S_c > K_c]$. Set $a_c = \lambda_c \partial \ell / \partial S_c \geq 0$. Summing over caps gives a gradient in $\text{span}\{\nabla_\theta S_c\}_c$, a subspace of dimension at most the number of caps.

Gradient clipping absorbs the scalar. Let $g = \nabla_\theta L_{CE} + \sum_{c'} a_{c'} \nabla_\theta S_{c'}$. In the limit where one cap’s term dominates and the remaining terms vanish, $g \rightarrow a_c \nabla_\theta S_c$ with $a_c > 0$; norm-clipping, when active ($\|g\| > \tau$), then returns $\tau g / \|g\| \rightarrow \tau \nabla_\theta S_c / \|\nabla_\theta S_c\|$, in which the positive scalar a_c (hence λ_c , ρ , and the shape of ℓ) cancels.

A finite multiplier suffices. Write $g(S) = \ell(\max(0, S-K); K, \rho)$. For $S < K$, $g \equiv 0$, so g has left-derivative 0 at K . For $S > K$, $g'(S) = K/(E+K)^2 + \frac{2\rho}{K} u/(1+u^2)^2$ with $E = S-K$ and $u = E/K$ (Eq. (2)); as $S \rightarrow K^+$ the first term $\rightarrow 1/K$ and the second $\rightarrow 0$, so g has right-derivative $1/K$ at K , independent of ρ . Parameterize a path along which the soft count rises through K , and let $c = |dL_{CE}/dS|_{S=K}$ be the cross-entropy pull at the cap; the constrained class is under-served at a cap $L < 100\%$, so $dL_{CE}/dS \leq 0$ there. The one-sided derivatives of $F = L_{CE} + \lambda g$ at K are $F'(K^-) = -c \leq 0$ and $F'(K^+) = -c + \lambda/K$. If $\lambda \geq cK$ then $F'(K^+) \geq 0$, so F does not decrease past the cap: $S=K$ is a local minimum along the path, and a finite multiplier attains the cap. The nonzero boundary slope $1/K$ is what makes the penalty exact, playing the role of the ℓ_1 penalty’s unit slope (Nocedal & Wright, 2006).

Figure 4 plots these two bounds against the violation size. Why does a hard count cap stress the dual-ascent baselines? On the paper grid the interaction is hidden: after the 50-epoch warmup the classifier is already saturated, and a saturation gate shared by every method (train accuracy ≥ 0.995 on two consecutive checks) stops cross-entropy updates within the first two constraint epochs, so the constraint phase is pure count optimization for all methods alike. We therefore probe with a one-epoch warmup, where the classifier keeps learning throughout the constraint phase and the two objectives visibly compete (Figure 5).

The probe shows three things. First, while the classifier is learning, the count cannot be forced down: each epoch’s cross-entropy updates re-inflate the constrained class, and the excess oscillates around its starting level for ~ 60 epochs under both methods (Fig. 5c). Dual ascent responds exactly as designed: the violation persists, so the multiplier grows every epoch, from 1 to 53 (seeds 2–3: 58, 60). In control terms this is integrator windup: the controller keeps integrating an error it has no ability to act on. Second, the escalation buys nothing. Every method takes a single gradient-clipped constraint step per epoch (max-norm 1.0, shared across methods), so extra multiplier cannot buy extra descent: the two cross-entropy traces are numerically indistinguishable (Fig. 5b), and the count comes down no faster at $\lambda=53$ than at TraLO’s ratcheted $\lambda \leq 0.19$. The asymmetry is in what the clip rail is doing. At $\lambda=53$ an unbounded-penalty gradient would dominate the update by orders of magnitude were it not clipped, whereas TraLO’s bounded penalty at $\lambda \leq 0.19$ demands orders of magnitude less force in the first place. Third, the count falls only once the classifier saturates and the gate stops CE updates (dotted lines in Fig. 5c), and then both methods reach the cap within a few epochs. The count outcome is identical; what differs is the pressure applied to the constrained class to get there.

The paper grid shows the same behavior in compressed form: the dual multipliers escalate while TraLO’s freezes (≤ 0.09 there), Fioretto’s native satisfaction is ~ 0.87 (0.84 on OctMNIST and TissueMNIST, 0.93 on DermMNIST), so its unbounded multiplier stalls short of the cap on $\sim 13\%$ of runs that never reach feasibility within the budget (Sec. 5.4); runs that end violated restore their best-satisfied checkpoint and remain competitive.

Nor is the escalation a step-size artifact. A dedicated fairness sweep (Block C of the released corpus; 96 runs on the hard-regime cells) varies Fioretto’s dual step over 0.001–0.01 and Hounie’s η over 0.001–0.1: Fioretto’s constrained-class F1 moves by less than 0.001 across the $10\times$ step range, larger Hounie steps only hurt ($0.43 \rightarrow 0.35$), and no setting closes the gap to TraLO on the OctMNIST tight cells (the best variant still trails by 0.019 cc-F1).

We state this mechanism *qualitatively only*. An audit shows that the magnitude of the escalation does not predict where TraLO wins (the multiplier is in fact largest in some cells where TraLO loses). So the claim is “dual ascent needs an unbounded multiplier plus a clip rail to hold a hard count, while TraLO’s bounded, frozen multiplier holds it alone,” not any dose-response of the form “more escalation $\Rightarrow$ bigger win.”

B Complete Component Ablation

This appendix is the source of the load-bearing finding of Sec. 6: of the four TraLO ingredients (saturating penalty, ratcheted-and-frozen multiplier, Adam-state reset, undershoot hinge), which two account for the constrained-class advantage.

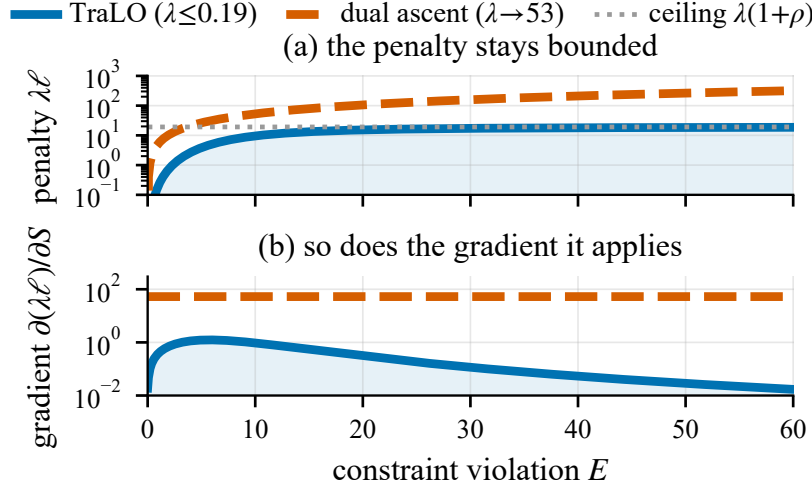


Figure 4: **Bounded vs. unbounded penalty and gradient.** At the multipliers each method reaches on the one-epoch probe, TraLO’s per-cap penalty (a) flattens at its ceiling $\lambda(1+\rho)$ and the gradient it applies (b) stays capped and decays far from the cap, while dual ascent’s linear penalty and constant $\lambda \approx 53$ gradient grow without bound in E and λ . This is the shape of the *objective*; whether the optimizer is sensitive to it is the empirical question the body answers: the scale is rescaled away, and the shape proves neutral by ablation and graft (App. B.2). Analytic, $K=10$.

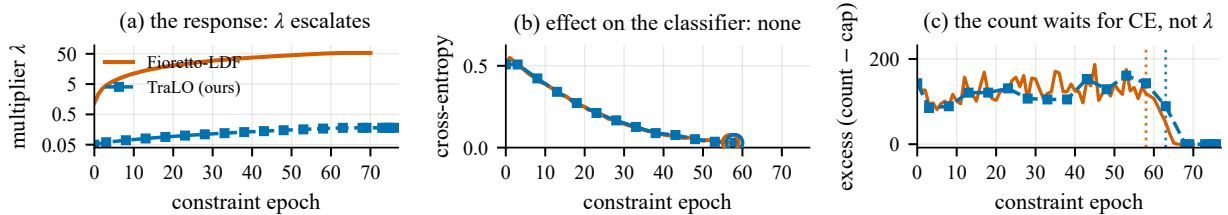


Figure 5: **The escalating multiplier is discarded, not used: a one-epoch-warmup probe** (Der-mMNIST $L50/G50$, RegNetY-400MF, seed 1). (a) Dual ascent integrates the standing violation into a multiplier that climbs to $\lambda=53$, while TraLO’s ratchet freezes at $\lambda \leq 0.19$. (b) The effect on the classifier is none: every method takes a single norm-clipped constraint step per epoch, so the two cross-entropy traces are numerically indistinguishable. (c) The count does not fall until cross-entropy finishes (train-accuracy gate, dotted), then reaches the cap for both. A $\sim 280\times$ gap in λ thus leaves training nearly identical: with the constraint term dominant and the step norm-clipped, the update is independent of λ . Adam’s invariance to a constant rescaling of the gradient (Kingma & Ba, 2015) points the same way, though we read *that* as an empirical observation rather than a theorem, since λ multiplies only the constraint term (so it shifts the cross-entropy-versus-count weighting) and the Adam cancellation is exact only as $\epsilon \rightarrow 0$ and where the constraint term dominates.

A within-cell ablation isolates which ingredients carry the hard-regime advantage: on the OctMNIST tight-cap cells where TraLO leads, we disable one component at a time and measure the paired change in constrained-class F1 (Table 2; $n=24$ per component). Two components are load-bearing. Removing the optimizer reset at satisfaction costs $+0.079$ cc-F1, the single largest effect, helping on all 24 paired runs and hence in all six cells (cell-level sign test $p=0.031$), because it breaks the post-satisfaction descent momentum that would otherwise erode the constrained class. Removing the undershoot hinge (the TraLO-bounded control, already in the frozen grid) costs $+0.036$ cc-F1 (positive on all 24 paired runs; cell-level sign test $p=0.031$; 0.019–0.070 across cells). The other components are cc-F1-neutral on these already-satisfied cells:

Table 2: **Complete component ablation on the OctMNIST hard-binding cells.** Effect on constrained-class F1 of removing each TraLO component in turn ($n=24$ paired runs each: 2 tight caps $\times$ 3 backbones $\times$ 4 seeds). Δ cc-F1 = full TraLO – (component removed); winrate = fraction of the 24 paired runs where removing the component hurts. Inference is at the cell level (the unit that does not repeat seeds): removing either load-bearing component hurts in all six cells (sign test $p=0.031$, the six-cell floor), while the ρ schedule and lambda freeze split the cells and are cc-F1-neutral on these already-satisfied cells. Two essential ingredients plus two neutral ones, *not* four co-equal components.

Component removed	Δ cc-F1	Winrate
<i>Load-bearing</i>		
Optimizer reset	+0.079	1.00
Undershoot hinge	+0.036	1.00
<i>Neutral</i>		
ρ schedule	+0.001	0.42
Lambda freeze	+0.000	0.38

the ρ schedule and the lambda freeze move it by under 0.001 (n.s.). A dedicated KL sweep on TissueMNIST (the $L30$ tight cap, MobileNetV3, $\alpha_{KL} \in \{0, 0.1, 0.3, 1.0\}$) confirms the anchor is off for good reason: $\alpha_{KL}=0$ (the paper recipe) gives the best constrained-class *and* macro-F1, while every nonzero anchor lowers cc-F1 (by 0.01–0.03) and only adds prediction edits. The KL term was a useful drift damper in an early version of the method, before the optimizer reset and undershoot hinge; once those two control post-satisfaction drift directly, the anchor is redundant. The advantage rests on the reset and the hinge. One caveat applies to all rows: the component importances are estimated inside the selected win region (the tight-cap cells where TraLO leads), so they quantify what carries that win, not what the components do elsewhere on the grid. The reset, ρ , and freeze legs are a dedicated ablation under the identical frozen recipe; the hinge leg is already in the main grid.

B.1 Grafting the two components onto the dual baselines

This subsection contains the graft evidence for the paper’s central mechanistic claim (Sec. 6): the reset and hinge, transplanted onto Fioretto-LDF and Hounie-RCL, close most of the tight-cap gap. It is the converse of the within-TraLO ablation above.

The ablation above removes components from TraLO; the converse test adds them to the baselines. We graft exactly the optimizer reset and the undershoot hinge (each using the host’s own multipliers and penalty) onto Fioretto-LDF and Hounie-RCL and rerun the six tight-cap cells: 2 caps $\times$ 3 backbones $\times$ 4 seeds, 144 runs in all, including re-run controls for TraLO and both hosts under identical warmup caches, the frozen recipe, and one GPU model. The re-run controls reproduce the main-grid values to ± 0.0013 in all 18 control cells, so the arms are compared on equal footing. Table 3 reports mean cc-F1 per cell.

The graft lifts Fioretto-LDF in six of six cells (+0.019 to +0.073 cc-F1) and Hounie-RCL in five of six (+0.035 to +0.103; the one exception, $L40$ on RegNet, drops by 0.028), recovering a median 86% of TraLO’s margin over the Fioretto host and 92% over the Hounie host (medians over the six cells, the loss cell included). Against each grafted host separately TraLO stays best-or-tied (8 wins, 4 ties, 0 losses in the twelve paired comparisons under the main-text win rule). Taken seed-by-seed against the stronger of the two grafted hosts, though, its mean edge is only +0.0015 and it trails in two of the six cells: once the recipe is grafted the three methods are statistically interchangeable, with what remains of TraLO’s lead concentrated on the ViT-B/16 (+0.015/+0.017 over the grafted Fioretto-LDF at the two caps). On macro-F1, TraLO versus the grafted Fioretto-LDF is a tie in all six cells.

As an anti-windup control, we also run Fioretto-LDF with dual restarts: all multipliers are reset to zero the moment the soft counts are feasible, following Gallego-Posada et al. (2022). Its deployed predictions are *identical* to plain Fioretto-LDF in every cell, although every run fires at least one restart. The reason is mechanistic: the deployed checkpoint is selected on the constraint-excess axis before first feasibility is reached,

and up to that epoch the two arms are step-for-step identical, so machinery that only alters post-feasibility dynamics cannot change the deployed model. TraLO beats this arm in all six cells. The dual baselines’ tight-cap deficit is therefore not an artifact of missing anti-windup machinery, and the linear-penalty comparison in the main grid is not a strawman in that sense.

B.2 The penalty shape is neutral

This subsection is the source of the paper’s negative penalty-shape finding (Sec. 6 and abstract): swapping Eq. (1)’s saturating penalty for a linear one, with the reset, hinge, ratchet, and freeze held fixed, leaves constrained-class predictions essentially unchanged.

The graft removes the question of whether TraLO’s *host* matters; a third arm removes the question of whether its *penalty* does. We rerun TraLO on the same six cells with Eq. (1)’s saturating penalty replaced by a linear one, $\ell = E/K$, holding the ratchet, the freeze, the optimizer reset and the undershoot hinge fixed, so the only change is the shape the method is named for. The arm is genuinely different, and by a wide margin: its logged constraint loss runs about $3\times$ smaller throughout. Its results are not: cc-F1 moves by -0.001 (the bounded shape wins 9 of 24 runs and loses 9, sign-test $p=1.0$; the six cell means split three positive and three negative, none beyond ± 0.006) and median time-to-feasibility by two constraint epochs. In 6 of the 24 runs the two arms land on *identical* cc-F1: changing the penalty shape changed no prediction at all, which the fixed-direction argument below anticipates.

This is what the analysis of Sec. 3 predicts, though the reason is structural rather than a generic property of Adam. The constraint penalty depends on the parameters only through the scalar soft-count S_c , so its gradient is $\ell'(S_c) \nabla_{\theta} S_c$: a scalar times a fixed direction. Multiplying the penalty by a positive constant scales only that scalar, and behind Adam’s coordinate-wise normalization $g \rightarrow cg$ sends $m \rightarrow cm$, $\sqrt{v} \rightarrow c\sqrt{v}$, leaving $\hat{m}/(\sqrt{v} + \epsilon)$ unchanged but for the ϵ term; the update along that direction is thus nearly scale-free. The multiplier is such a scalar, so an escalating one cannot by itself reach the weights: in the escalation probe of App. A, where $\lambda=53$ (nearly $280\times$ the probe’s 0.19) drives $\|g\|>1$, it is the max-norm rail that caps the step, and the count falls no faster than at $\lambda \leq 0.19$. The penalty *shape* and the weight ρ reshape $\ell'(S_c)$ rather than rescale it, so they are not covered by the constant- c argument; we establish their neutrality empirically instead, by graft (shape) and by the ablation rows of Table 2 (ρ schedule, freeze). What no rescaling touches is Adam’s own accumulated state, and the hinge, which reverses the direction to $-\nabla_{\theta} S_c$ while the count undershoots: these are the components the ablation ranks first and the graft transfers.

We report this as a negative result about our own formulation. Eq. (1) earns its place by keeping the multiplier small and readable, and by making the two-phase argument of Sec. 3 available at all; it does not earn the constrained-class gains, which belong to the reset and the hinge.

B.3 TraLO’s own hyperparameters are not doing the work

This subsection answers a natural robustness question: are TraLO’s own tunable knobs (the hinge weight β and the ratchet step λ_{step}) what deliver the OctMNIST tight-cap advantage of Sec. 5.1? They are not.

The dual baselines’ step size is swept an order of magnitude to confirm their tuning is not what wins or loses the comparison (App. A); fairness asks the same of TraLO’s own continuous knobs, the more so in a transductive setting with no held-out split. On the six OctMNIST tight cells we sweep the hinge weight $\beta \in \{0.25, 0.5, 0.75, 1.0\}$ and the ratchet step $\lambda_{\text{step}} \in \{0.001, 0.002, 0.004\}$ around the frozen recipe ($\beta=0.5$, $\lambda_{\text{step}}=0.002$), one knob at a time, 24 runs per arm, with everything else, warmup cache included, held fixed (120 runs).

The ratchet step is neutral. Moving λ_{step} over a factor of four shifts mean cc-F1 by at most 0.0012 and the median time-to-feasibility by under an epoch. This is the scalar-rescaling argument of Sec. 3 made good: λ_{step} sets only how fast the multiplier climbs, and the multiplier is a positive scalar on the fixed soft-count direction, which Adam’s normalization and the clip rail absorb.

The hinge weight matters, but the advantage is not a knife-edge at $\beta=0.5$. As the one continuous knob on a load-bearing component, β does move cc-F1, but by at most 0.003 in the six-cell mean across

Cap	Backbone	Fioretto-LDF			Hounie-RCL		
		host	+R+H	+restart	host	+R+H	TraLO
L30	MNetV3	0.391	0.412	0.391	0.326	0.411	0.407
	RegNet	0.372	0.399	0.372	0.299	0.402	0.407
	ViT-B/16	0.335	0.409	0.335	0.332	0.416	0.423
L40	MNetV3	0.487	0.510	0.487	0.456	0.491	0.509
	RegNet	0.452	0.470	0.452	0.410	0.383	0.473
	ViT-B/16	0.462	0.496	0.462	0.431	0.506	0.513

Table 3: **Graft and anti-windup arms on the OctMNIST tight caps** (mean cc-F1 over seeds 1–4; **bold** = best in row). +R+H grafts TraLO’s optimizer reset and undershoot hinge onto the host; +restart resets all dual multipliers to zero at soft feasibility (Gallego-Posada et al., 2022). All 144 runs share the frozen recipe, warmup caches, and GPU model with the re-run hosts; the re-run hosts reproduce the main-grid values to ± 0.0013 .

the whole $\{0.25, \dots, 1.0\}$ range, smallest at the weakest hinge $\beta=0.25$, exactly where less hinge restores less constrained-class recall. More to the point, TraLO clears the best trained baseline in all 30 arm-by-cell comparisons, with margins from +0.014 to +0.088: the tight-cap advantage survives every setting of the two knobs, not only the values we fixed a priori.

B.4 The convergence ordering is not an initialization artifact

This subsection rules out that TraLO’s faster feasibility over the dual baselines (Sec. 5.4) is an artifact of its non-zero multiplier initialization, by rerunning each dual baseline from TraLO’s own start value.

TraLO starts its multiplier at $\lambda=0.05$ while the dual baselines start at $\lambda=0$ and integrate upward, so one could read TraLO’s faster feasibility (Sec. 5.4) as a head start rather than a property of the method. We test this on the six OctMNIST tight cells by rerunning each dual baseline from TraLO’s own $\lambda=0.05$ start, changing nothing else (24 runs each). The nonzero start leaves Fioretto-LDF’s median time-to-feasibility unchanged (28 constraint epochs, taken over the runs that reach feasibility) and it still fails to reach the count natively on 6 of 24 runs; it narrows Hounie-RCL’s median from 40 to 31 (Hounie-RCL reaches feasibility on every run, so its median is over all 24). Both still trail TraLO’s 25. So the head start explains part of Hounie’s gap and little of Fioretto’s, and closes neither: on these cells the ratchet-and-freeze schedule reaches and holds the count faster than either a zero- or a nonzero-start dual ascent. This control is specific to the OctMNIST tight cells; the cross-dataset census of Sec. 5.4 uses each method as published.

C Native Satisfaction: a Deployment Property, not a Headline

This appendix documents the deployment properties (native satisfaction and prediction flips) that back the macro-F1 advantage over post-hoc clipping of Sec. 5.3. It is deliberately separated from the main quality claims because, on its own, native satisfaction does not discriminate methods: every deployable system meets a hard count one way or another.

We separate *native satisfaction* (meeting the count with no post-hoc edit) from the quality results in the main paper because, on its own, it does not discriminate methods: every deployable system meets a hard count one way or another, and post-hoc clipping reaches satisfaction 1.00 by construction, simply by editing predictions. We report it only to document the mechanism behind TraLO’s quality advantage over clipping (Sec. 5.3): TraLO reaches the count during training and so never invokes the editing step that costs clipping its accuracy. Table 4 gives the deployment costs at the tight L30 cap on MobileNetV3: macro-F1, accuracy, and the number of predictions each method must overwrite (Flips). Figure 6 shows native satisfaction per method over the full grid. TraLO and Hounie-RCL meet the count in every run; Fioretto-LDF and the hinge-free TraLO-bounded do most of the time; the post-hoc clippers meet it natively only at loose caps. A caveat on *flips*: at near-trivial loose caps (e.g. OctMNIST L80/L90) TraLO can record more flips than

Table 4: **Model quality at the tight L30 cap** (MobileNetV3; each value is the mean over seeds 1–4 *at this single cap level*). All methods meet the count; the post-hoc clippers do so by *editing predictions*, and Flips is the number of test predictions overwritten to enforce the cap. This editing erodes **macro-F1**, the imbalance-aware quality metric, on every dataset. Raw *accuracy* is nearly blind to it: the constrained class is rare, so clipping it barely moves the aggregate and the clippers can tie or even beat the trained methods on accuracy while losing macro-F1. This is why F1, not accuracy, is the right quality metric here. [†] = post-hoc clipping. **Bold** = best, underline = 2nd-best per column within each dataset block.

Method	Tissue			Derm			OctMNIST		
	macro-F1 $\uparrow$	Acc. $\uparrow$	Flips $\downarrow$	macro-F1 $\uparrow$	Acc. $\uparrow$	Flips $\downarrow$	macro-F1 $\uparrow$	Acc. $\uparrow$	Flips $\downarrow$
Heur. [†]	0.347	0.478	99.5	0.722	0.846	136.5	0.666	0.695	109.5
LP-LG [†]	0.347	0.478	99.5	0.721	0.846	136.5	0.665	0.695	109.5
Fioretto	0.405	0.536	<u>7.8</u>	0.723	0.842	10.2	<u>0.701</u>	0.738	<u>0.2</u>
Hounie	<u>0.406</u>	<u>0.537</u>	11.2	0.728	0.843	19.2	<u>0.678</u>	0.723	0.0
TraLO-b	0.405	0.536	10.0	<u>0.727</u>	<u>0.844</u>	<u>9.8</u>	0.700	0.738	0.0
TraLO	0.408	0.539	7.0	0.726	<u>0.844</u>	1.0	0.702	<u>0.736</u>	0.0

Table 5: **TraLO’s macro-F1 advantage over post-hoc clipping is backbone- and dataset-general.** Mean paired macro-F1 gap of TraLO over the best post-hoc clipper (per-seed best of Heuristic/LP-LG), averaged over the symmetric caps for each dataset–backbone cell; win/tie/loss counts the caps. The mean gain is positive in every cell, largest on TissueMNIST; it also holds under asymmetric caps (App. D).

Backbone	Tissue	Derm	OctMNIST
MobileNetV3	+0.058 9/0/0	+0.009 6/2/1	+0.044 9/0/0
RegNetY-400MF	+0.082 9/0/0	+0.005 6/2/1	+0.011 6/3/0
ViT-B/16	+0.020 9/0/0	+0.033 9/0/0	+0.007 5/4/0
MobileNetV2	+0.049 6/1/0	+0.004 3/3/0	+0.040 5/0/0

a clipper that barely has to act, so the robust statement is “satisfies natively while preserving macro-F1,” never a raw flip count.

The macro-F1 advantage over clipping is not specific to MobileNetV3 or to the single L30 cap of Table 4. Table 5 averages the paired gap over the symmetric caps for each dataset–backbone cell, adding MobileNetV2 to the three main-grid backbones: the mean gain is positive in every one of the twelve cells, largest on TissueMNIST, and it persists under asymmetric caps (App. D).

D Per-Cell Results Across the Full Grid

This appendix is the per-cell substrate of the regime map claim (Sec. 5.2): every dataset×backbone×cap cell in the paper final grid is listed with its paired gaps and win/tie/loss classification, so that any regime-level assertion in the body can be verified directly against the individual cells behind it.

Tables 9–12 give the fully granular breakdown behind the regime summary of Sec. 5.2: one row per (dataset, backbone, cap), averaged over the four seeds *only*, with no pooling across backbones or caps. Each row pairs TraLO against the best constraint-trained baseline on constrained-class F1 (per-seed best of Fioretto-LDF/Hounie-RCL) and against the best post-hoc clipper on macro-F1, both paired by seed.

Two patterns read straight off the cells. The constrained-class advantage is *consistent* (all three backbones, every seed) only at the OctMNIST tight caps (L30/L40), where it is largest on ViT-B/16; a handful of isolated single-backbone cells elsewhere also clear the +0.005 bar, but no other regime wins across backbones, and none is a loss beyond seed noise. The macro-F1 margin over clipping is positive in 77 of the 81 symmetric cells; the four exceptions are the very tightest L10/L20 caps, where the cap nearly zeroes out the constrained class and the gap is within ± 0.02 . The asymmetric caps (Table 12) hold the *preliminary* constrained-class

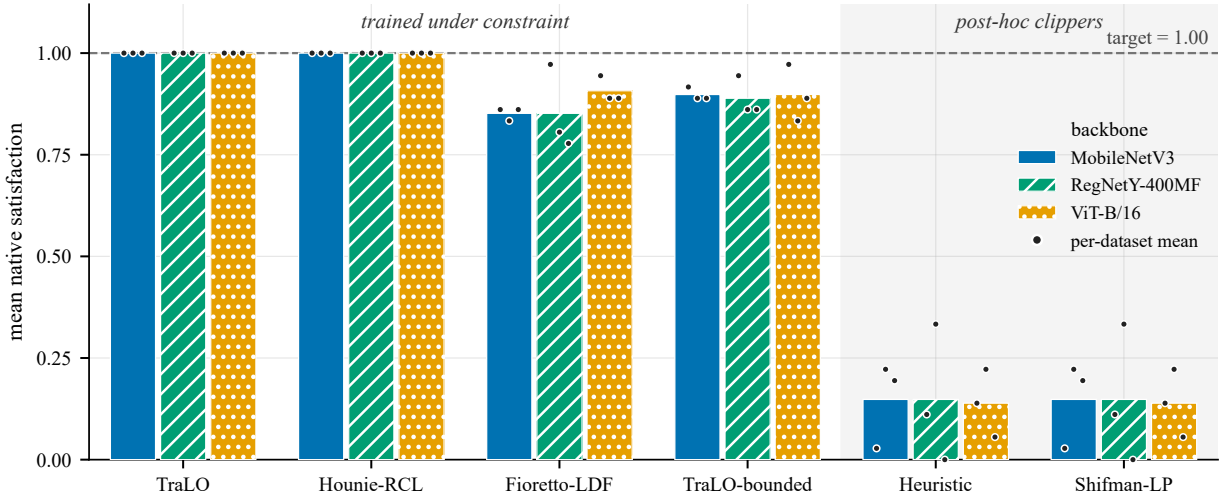


Figure 6: **Methods trained under the constraint satisfy the count natively; post-hoc clippers do not.** Mean native satisfaction per method and backbone over the paper_final grid ($N=108$ runs per method $\times$ backbone: 3 datasets $\times$ 9 caps $\times$ 4 seeds); the dots are the three per-dataset means. TraLO and Hounie-RCL meet the count in every run (satisfaction 1.00); Fioretto-LDF and TraLO-bounded reach it most of the time (~ 0.85 – 0.91); the post-hoc clippers satisfy natively only at loose caps (~ 0.14 – 0.15) and otherwise reach the cap solely by editing predictions, the source of their accuracy tax (Table 4).

Table 6: **The constrained-class picture is backbone-general.** Mean paired constrained-class-F1 gap of TraLO over the best constraint-trained baseline (per-seed best of Fioretto-LDF/Hounie-RCL), averaged over the symmetric caps for each dataset–backbone cell; win/tie/loss counts the caps (a win needs the mean gap and at least half the seeds to clear $+0.005$; loss is the mirror). Across all four backbones (the three main-grid ones plus MobileNetV2), TissueMNIST and DermMNIST show no systematic constrained-class advantage (per-backbone means within ± 0.003 , mostly ties), while OctMNIST is a positive margin on every backbone. The honest tie and the OctMNIST win are both properties of the regime, not of one architecture.

Backbone	Tissue	Derm	OctMNIST
MobileNetV3	−0.002 0/6/3	+0.002 0/9/0	+0.004 2/6/1
RegNetY-400MF	−0.002 1/5/3	−0.003 0/7/2	+0.011 4/5/0
ViT-B/16	−0.001 1/7/1	−0.002 0/7/2	+0.018 5/4/0
MobileNetV2	+0.002 3/3/1	−0.002 1/3/2	+0.002 1/3/1

win, on DermMNIST where the global cap is tighter than the group cap. That per-cell table is MobileNetV3, the only backbone with the full asymmetric grid; a cross-backbone check on the four shared $G < L$ cap pairs (Table 8) shows the constrained-class advantage reproduces on RegNet (+0.9pp, 75% seed-wins) and ties on the ViT (−0.0pp), while the macro-F1 advantage over clipping holds on all three backbones. We report the asymmetric result as preliminary because only MobileNetV3 has been run on the complete asymmetric grid, not because a single backbone shows it.

Table 6 summarizes the constrained-class picture across backbones, adding MobileNetV2 to the three main-grid backbones and averaging the paired gap over the symmetric caps: on TissueMNIST and DermMNIST no backbone shows a systematic advantage (means within ± 0.003), while OctMNIST is positive on all four. The honest tie and the OctMNIST advantage are each backbone-general, not artifacts of one architecture. Table 7 isolates the OctMNIST tight caps ($L30/L40$) behind that positive row: the paired delta over the best trained dual is positive at every backbone $\times$ cap cell on cc-F1 (all four seeds) and improves macro-F1 at five of the six cells, peaking on the ViT-B/16. This is the per-backbone breakdown Sec. 5.1 cites next to its consolidated grid (Table 1).

Table 7: **The OctMNIST tight-cap win is backbone-general, largest on the ViT-B/16, and carries to overall accuracy.** Paired gap of TraLO over the best trained dual (per-seed best of Fioretto-LDF/Hounie-RCL) at the tight caps, mean over seeds 1–4 ($\pm$ across-seed std). Constrained-class F1 leads at every cell on all 4/4 seeds; macro-F1 improves at five of the six cells (the exception a wash at MobileNetV3 $\times$ L30), again peaking at the ViT-B/16.

Backbone	Cap	Δ cc-F1	Δ macro-F1
MobileNetV3	L30	+0.016 $\pm$.014	+0.001 $\pm$.001
	L40	+0.022 $\pm$.029	+0.008 $\pm$.013
RegNetY-400MF	L30	+0.035 $\pm$.006	+0.013 $\pm$.005
	L40	+0.021 $\pm$.003	+0.007 $\pm$.002
ViT-B/16	L30	+0.081 $\pm$.045	+0.028 $\pm$.014
	L40	+0.051 $\pm$.019	+0.016 $\pm$.008

Table 8: **The DermMNIST global-tighter ($G < L$) win reproduces across backbones.** Cross-backbone check on the four shared $G < L$ cap pairs ($L50/G20$, $L50/G30$, $L70/G50$, $L80/G70$; mean over seeds 1–4, $n=16$ paired seed-cells). Δ cc-F1 is TraLO minus the best constraint-trained baseline; Δ macro-F1 is TraLO minus the best post-hoc clipper; both paired by seed, with *win* the seed-cell fraction favoring TraLO. The constrained-class advantage holds on MobileNetV3 and RegNet and ties on the ViT; the macro-F1 advantage over clipping holds on all three.

Backbone	Δ cc-F1 vs. trained		Δ macro-F1 vs. clipper	
	Δ	win	Δ	win
MobileNetV3	+0.012	81%	+0.015	77%
RegNetY-400MF	+0.009	75%	+0.002	50%
ViT-B/16	−0.000	38%	+0.032	100%

E Datasets and Group Construction

This appendix documents the class distributions and the group construction that back the constrained-class and per-group cap definitions of Sec. 4. It also elaborates on the ceiling-versus-floor caveat that motivates the fairness scoping in the Broader Impact statement.

The constrained class is a genuine minority on two of the three datasets and not on the third: GE is 7.1% of the TissueMNIST test set (171/2400) and melanoma is 11.1% of DermMNIST (223/2003), while OctMNIST’s test split is class-balanced by construction (250 per class), making drusen 25%. We keep OctMNIST because it is where the cap binds hardest, not because it matches the screening story: at $L30$ the cap admits 75 of 250 true drusen, so a structural majority of them cannot be flagged whatever the model does. Readers should not take the OctMNIST result as evidence about rare-class screening, and a clinical deployment would more plausibly cap the urgent classes (CNV, DME) than drusen.

Local caps are only over real groups on DermMNIST. There we pool HAM10000’s `localization` field into three anatomical groups: torso (back, trunk, abdomen, chest, genital, unknown), extremity (upper/lower extremity, foot, hand, acral), and head/neck (face, neck, scalp, ear), giving 1030/738/235 test images carrying 98/85/40 melanomas. The groups are deliberately unequal in both size and prevalence, which is what makes the per-group caps bind differently.

TissueMNIST and OctMNIST carry no group metadata, so their local caps use a fixed synthetic binary split drawn once from a seeded generator and stored with the data. These groups are arbitrary by construction. They exercise the mechanism, several simultaneous caps that must hold at once alongside the global cap, and nothing else: no fairness reading of the TissueMNIST or OctMNIST local caps is available, and we do not offer one. Even on DermMNIST, where the groups are real, a local cap is a per-group *ceiling* on flagging melanoma rather than a floor on detecting it, so satisfying it is not evidence of equitable treatment. A deployment that set these caps from estimated prevalence would inherit whatever under-diagnosis those

Table 9: **TissueMNIST: per-cell constrained-class and overall F1.** One row per (backbone, cap), mean over seeds 1–4. cc-F1 columns compare TraLO to the best constraint-trained baseline (per-seed best of Fioretto-LDF/Hounie-RCL, paired); *win* is the seed-winrate. Δ_{mac} is TraLO minus the best post-hoc clipper on macro-F1. **All Δ are in units of 10^{-3}** (so +13.4 means +0.0134 F1). Each cell is scored *win* ($\Delta \geq +5$ on $\geq 3/4$ seeds), *loss* (the mirror image), or *tie*; the per-backbone tally is given in each block header. Read this table for the *pattern*, not the individual cells: on the constrained class it is overwhelmingly ties, which is the honest result, while Δ_{mac} stays positive nearly everywhere: the constraint-trained family is better *overall* than post-hoc clipping even where it merely ties on the constrained class.

Cap	cc-F1 vs. best trained				macro	
	best	TraLO	Δ	win		Δ_{mac}
<i>MobileNetV3</i> 9 cells: 0W / 6T / 3L						
L10	0.106	0.101	-5.3	1/4	loss	+28.3
L20	0.205	0.205	0.0	1/4	tie	+53.8
L30	0.279	0.275	-4.5	1/4	tie	+61.0
L40	0.301	0.300	-0.9	2/4	tie	+59.6
L50	0.352	0.354	+1.9	2/4	tie	+59.9
L60	0.381	0.375	-5.5	0/4	loss	+64.1
L70	0.397	0.399	+1.8	2/4	tie	+64.1
L80	0.414	0.407	-6.5	1/4	loss	+64.7
L90	0.409	0.409	0.0	1/4	tie	+64.6
<i>RegNetY-400MF</i> 9 cells: 0W / 6T / 3L						
L10	0.109	0.112	+2.7	1/4	tie	+61.1
L20	0.212	0.195	-17.1	0/4	loss	+73.2
L30	0.291	0.273	-18.0	0/4	loss	+79.7
L40	0.314	0.324	+9.9	2/4	tie	+91.4
L50	0.362	0.366	+3.9	3/4	tie	+84.8
L60	0.379	0.383	+3.7	2/4	tie	+83.7
L70	0.414	0.409	-5.2	1/4	loss	+91.4
L80	0.429	0.432	+3.3	2/4	tie	+89.3
L90	0.435	0.431	-4.6	1/4	tie	+86.9
<i>ViT-B/16</i> 9 cells: 1W / 7T / 1L						
L10	0.133	0.122	-10.7	0/4	loss	+11.5
L20	0.239	0.239	0.0	0/4	tie	+15.6
L30	0.327	0.327	0.0	0/4	tie	+18.0
L40	0.360	0.373	+13.0	4/4	win	+19.5
L50	0.422	0.418	-3.9	0/4	tie	+23.7
L60	0.445	0.447	+1.8	1/4	tie	+22.5
L70	0.466	0.462	-3.4	1/4	tie	+20.5
L80	0.487	0.482	-4.9	2/4	tie	+25.4
L90	0.495	0.497	+1.5	2/4	tie	+24.6

estimates already encode. We report the mechanism and leave the allocation question, which is a question about who sets K_g and on what evidence, outside our claims.

Table 10: **DermMNIST: per-cell constrained-class and overall F1.** One row per (backbone, cap), mean over seeds 1–4. cc-F1 columns compare TraLO to the best constraint-trained baseline (per-seed best of Fioretto-LDF/Hounie-RCL, paired); *win* is the seed-winrate. Δ_{mac} is TraLO minus the best post-hoc clipper on macro-F1. **All Δ are in units of 10^{-3}** (so +13.4 means +0.0134 F1). Each cell is scored *win* ($\Delta \geq +5$ on $\geq 3/4$ seeds), *loss* (the mirror image), or *tie*; the per-backbone tally is given in each block header. Read this table for the *pattern*, not the individual cells: on the constrained class it is overwhelmingly ties, which is the honest result, while Δ_{mac} stays positive nearly everywhere: the constraint-trained family is better *overall* than post-hoc clipping even where it merely ties on the constrained class.

Cap	cc-F1 vs. best trained					macro
	best	TraLO	Δ	win		Δ_{mac}
<i>MobileNetV3</i> 9 cells: 0W / 9T / 0L						
L10	0.169	0.169	0.0	1/4	tie	-18.0
L20	0.310	0.312	+1.9	1/4	tie	-0.8
L30	0.417	0.419	+1.7	1/4	tie	+4.5
L40	0.506	0.510	+3.2	2/4	tie	+11.6
L50	0.563	0.564	+1.5	1/4	tie	+13.6
L60	0.618	0.618	0.0	2/4	tie	+12.8
L70	0.645	0.648	+2.6	3/4	tie	+16.8
L80	0.655	0.660	+5.0	2/4	tie	+21.9
L90	0.663	0.668	+4.7	3/4	tie	+19.5
<i>RegNetY-400MF</i> 9 cells: 0W / 7T / 2L						
L10	0.178	0.178	0.0	0/4	tie	-12.4
L20	0.317	0.317	0.0	1/4	tie	+4.3
L30	0.421	0.424	+3.5	3/4	tie	+5.1
L40	0.502	0.500	-1.6	0/4	tie	+5.7
L50	0.545	0.543	-1.5	1/4	tie	+3.7
L60	0.598	0.588	-9.8	0/4	loss	+9.8
L70	0.623	0.612	-10.6	1/4	loss	+7.7
L80	0.640	0.637	-2.5	1/4	tie	+11.1
L90	0.649	0.647	-2.4	1/4	tie	+11.9
<i>ViT-B/16</i> 9 cells: 0W / 7T / 2L						
L10	0.169	0.165	-4.1	0/4	tie	+21.2
L20	0.300	0.293	-7.4	0/4	loss	+28.6
L30	0.388	0.386	-1.7	0/4	tie	+29.9
L40	0.460	0.460	0.0	1/4	tie	+34.3
L50	0.510	0.507	-3.0	0/4	tie	+36.2
L60	0.546	0.546	0.0	1/4	tie	+35.1
L70	0.575	0.575	0.0	2/4	tie	+28.4
L80	0.610	0.604	-6.2	0/4	loss	+37.5
L90	0.606	0.612	+5.9	2/4	tie	+43.2

Table 11: **OctMNIST: per-cell constrained-class and overall F1.** One row per (backbone, cap), mean over seeds 1–4. cc-F1 columns compare TraLO to the best constraint-trained baseline (per-seed best of Fioretto-LDF/Hounie-RCL, paired); *win* is the seed-winrate. Δ_{mac} is TraLO minus the best post-hoc clipper on macro-F1. **All Δ are in units of 10^{-3}** (so +13.4 means +0.0134 F1). Each cell is scored *win* ($\Delta \geq +5$ on $\geq 3/4$ seeds), *loss* (the mirror image), or *tie*; the per-backbone tally is given in each block header. Read this table for the *pattern*, not the individual cells: on the constrained class it is overwhelmingly ties, which is the honest result, while Δ_{mac} stays positive nearly everywhere: the constraint-trained family is better *overall* than post-hoc clipping even where it merely ties on the constrained class.

Cap	cc-F1 vs. best trained					macro
	best	TraLO	Δ	win		Δ_{mac}
<i>MobileNetV3</i> 9 cells: 2W / 6T / 1L						
L10	0.176	0.178	+1.8	1/4	tie	+36.7
L20	0.312	0.312	0.0	1/4	tie	+43.6
L30	0.391	0.407	+16.1	4/4	win	+36.4
L40	0.487	0.509	+21.9	4/4	win	+52.6
L50	0.581	0.583	+1.4	2/4	tie	+50.2
L60	0.635	0.639	+3.8	2/4	tie	+49.1
L70	0.658	0.650	-8.3	1/4	loss	+40.6
L80	0.676	0.677	+1.1	1/4	tie	+42.2
L90	0.689	0.692	+2.1	1/4	tie	+45.0
<i>RegNetY-400MF</i> 9 cells: 5W / 4T / 0L						
L10	0.171	0.173	+1.8	1/4	tie	-0.9
L20	0.308	0.308	0.0	1/4	tie	+6.6
L30	0.372	0.407	+35.3	4/4	win	+6.4
L40	0.452	0.473	+20.9	4/4	win	+5.7
L50	0.545	0.544	-1.3	1/4	tie	+10.7
L60	0.583	0.603	+20.1	3/4	win	+15.7
L70	0.626	0.645	+18.9	4/4	win	+18.3
L80	0.647	0.652	+5.6	3/4	win	+14.0
L90	0.672	0.671	-1.1	1/4	tie	+18.0
<i>ViT-B/16</i> 9 cells: 5W / 4T / 0L						
L10	0.176	0.175	-1.8	0/4	tie	+0.3
L20	0.313	0.318	+5.0	2/4	tie	+2.5
L30	0.341	0.422	+81.0	4/4	win	+4.1
L40	0.462	0.512	+50.7	4/4	win	+4.5
L50	0.592	0.597	+5.3	4/4	win	+9.8
L60	0.638	0.643	+5.0	3/4	win	+7.2
L70	0.671	0.672	+1.2	1/4	tie	+10.2
L80	0.697	0.701	+4.5	2/4	tie	+12.9
L90	0.701	0.714	+12.6	4/4	win	+11.4

Table 12: **Asymmetric caps (MobileNetV3): per-cell F1.** Group cap L and global cap G set independently; one row per cap pair, mean over seeds 1–4. Columns, 10^{-3} units, and the win/tie/loss rule are as in Table 10. The contrast between the two DermMNIST blocks is the point: the preliminary cc-F1 win sits in the global-tighter ($G < L$) block and vanishes when the local cap binds instead ($G > L$), which is the regime dependence the main paper reports; Δ_{mac} stays positive throughout.

L/G	cc-F1 vs. best trained				macro	
	best	TraLO	Δ	win	Δ_{mac}	
<i>DermMNIST</i> , $G < L$ (<i>global tighter</i>)						10 cells: 5W / 4T / 1L
30/20	0.296	0.313	+17.1	3/4	win	-1.1
50/20	0.295	0.314	+19.0	3/4	win	+29.2
50/30	0.411	0.417	+6.2	2/4	tie	+1.9
70/20	0.296	0.306	+9.5	2/4	tie	+6.3
70/30	0.413	0.417	+4.5	2/4	tie	+6.3
70/50	0.554	0.565	+11.3	4/4	win	+16.8
80/20	0.295	0.287	-7.5	1/4	loss	+2.9
80/30	0.411	0.410	-1.3	1/4	tie	+33.0
80/50	0.553	0.563	+9.6	4/4	win	+14.6
80/70	0.639	0.650	+10.1	4/4	win	+23.8
<i>DermMNIST</i> , $G > L$ (<i>local tighter</i>)						10 cells: 0W / 8T / 2L
20/30	0.312	0.312	0.0	1/4	tie	+3.1
20/50	0.312	0.310	-1.9	1/4	tie	+28.5
20/70	0.312	0.310	-1.9	0/4	tie	+0.5
20/80	0.312	0.310	-1.9	0/4	tie	+2.8
30/50	0.416	0.417	+1.7	2/4	tie	+12.2
30/70	0.417	0.416	-1.7	1/4	tie	+9.8
30/80	0.417	0.417	0.0	1/4	tie	+17.7
50/70	0.567	0.561	-6.0	0/4	loss	+16.3
50/80	0.564	0.558	-6.0	0/4	loss	+18.1
70/80	0.645	0.646	+1.3	3/4	tie	+25.0
<i>TissueMNIST</i> , $G < L$ (<i>global tighter</i>)						10 cells: 2W / 4T / 4L
30/20	0.171	0.172	+0.8	1/4	tie	+15.1
50/20	0.177	0.166	-11.3	0/4	loss	+22.7
50/30	0.235	0.249	+14.1	3/4	win	+15.4
70/20	0.179	0.166	-13.6	0/4	loss	+14.0
70/30	0.236	0.216	-20.8	0/4	loss	+10.7
70/50	0.297	0.297	+0.3	2/4	tie	+15.5
80/20	0.179	0.164	-15.3	0/4	loss	+13.0
80/30	0.238	0.245	+7.4	3/4	win	+25.6
80/50	0.298	0.292	-5.8	2/4	tie	+12.3
80/70	0.324	0.329	+4.9	2/4	tie	+15.8
<i>TissueMNIST</i> , $G > L$ (<i>local tighter</i>)						10 cells: 2W / 7T / 1L
20/30	0.188	0.185	-2.5	0/4	tie	+11.3
20/50	0.190	0.178	-12.2	0/4	loss	+24.3
20/70	0.188	0.185	-2.5	1/4	tie	+11.9
20/80	0.188	0.183	-4.9	1/4	tie	+8.5
30/50	0.243	0.243	0.0	2/4	tie	+17.9
30/70	0.243	0.241	-2.2	1/4	tie	+15.8
30/80	0.243	0.243	0.0	1/4	tie	+25.6
50/70	0.303	0.317	+13.6	3/4	win	+19.1
50/80	0.298	0.313	+15.6	3/4	win	+16.4
70/80	0.340	0.345	+5.1	2/4	tie	+18.0